

MASTER OF COMPUTER APPLICATIONS

III SEMESTER

Data Science

Prepared by-

Prof.Sachin Zurange

-

--:INDEX:--

Topic No.	Topic Name	Page No
1	Introduction to Data Science	3
2	Mathematics & Statistics for Data Science	4
3	Programming for Data Science	12
4	Data Wrangling and Preprocessing & Exploratory Data Analysis (EDA)	95
5	Model Development & Evaluation & Deployment	122

Unit I: Introduction to Data Science

1.1 What is Data Science

Data Science is an interdisciplinary field that uses scientific methods, algorithms, processes, and systems to extract knowledge and insights from structured and unstructured data. It combines elements of statistics, computer science, mathematics, domain expertise, and data visualization to understand and solve real-world problems using data.

1.2 Importance of Data Science in modern industries

.Informed Decision-Making

Data science provides actionable insights by analyzing vast amounts of data. This helps industries make informed decisions rather than relying on intuition or guesswork.

- Example: Retail companies use sales data to predict demand and optimize inventory.

2. Improved Efficiency and Productivity

By identifying bottlenecks and inefficiencies through data analysis, businesses can streamline processes and improve performance.

- Example: Manufacturing industries use predictive maintenance to reduce machine downtime.

3. Personalized Customer Experiences

Data science enables personalized marketing, recommendations, and customer service by analyzing customer behavior and preferences.

- Example: E-commerce platforms like Amazon and Netflix use recommendation algorithms to suggest products and content.

4. Risk Management and Fraud Detection

Industries can detect fraud patterns, assess risks, and implement preventive measures using data models and machine learning.

- Example: Banks and financial institutions use data science to detect unusual transactions and prevent fraud.

5. Innovation and New Product Development

By understanding market trends and customer feedback, companies can innovate and design better products and services.

- Example: Automotive industries use customer feedback and sensor data to design safer and more efficient vehicles.

6. Healthcare Advancements

In healthcare, data science improves diagnosis, treatment plans, and patient outcomes by analyzing medical records and research data.

- Example: Predictive models are used to forecast disease outbreaks and personalize treatments.

7. Competitive Advantage

Organizations that leverage data science can outperform competitors by being more agile, customer-focused, and innovative.

- Example: Businesses that adopt data-driven marketing see higher conversion rates than those using traditional methods.

8. Operational Forecasting

Data science helps predict future trends, such as sales, weather, or demand, allowing better strategic planning.

- Example: Airlines use data science for dynamic pricing and route optimization.

1.3 Key roles in data science

1. Data Scientist

- **Responsibilities:**
 - Extract insights from structured and unstructured data.
 - Build predictive models using machine learning.
 - Communicate findings through visualizations and reports.
- **Skills:** Python, R, SQL, statistics, machine learning, data visualization tools (Tableau, Power BI, Matplotlib, etc.).

2. Data Analyst

- **Responsibilities:**
 - Analyze data sets to identify trends and patterns.
 - Create dashboards and reports for business users.
 - Assist in decision-making through data-driven insights.
- **Skills:** Excel, SQL, Python (basic), Tableau/Power BI, statistical analysis.

3. Data Engineer

- **Responsibilities:**
 - Design, build, and maintain data pipelines and infrastructure.
 - Prepare data for analysis by cleaning and transforming raw data.
-

- Ensure data quality and consistency.
- **Skills:** SQL, Python/Scala/Java, ETL tools, cloud platforms (AWS, Azure, GCP), big data tools (Hadoop, Spark).

4. Machine Learning Engineer

- **Responsibilities:**
 - Design and deploy machine learning models at scale.
 - Optimize model performance and integrate with production systems.
 - Work closely with data scientists and engineers.
- **Skills:** Python, TensorFlow/PyTorch, ML algorithms, model deployment (Flask, Docker, cloud services).

5. Business Analyst

- **Responsibilities:**
 - Translate business requirements into data problems.
 - Use data to support strategic decision-making.
 - Communicate insights to non-technical stakeholders.
- **Skills:** Business acumen, Excel, SQL, basic data visualization.

6. Data Architect

- **Responsibilities:**
-

- Design the overall data architecture for an organization.
 - Ensure that data is organized, secured, and scalable.
 - Choose appropriate database and data storage technologies.
- **Skills:** Database design, data modeling, cloud data services, metadata management.

7. Statistician

- **Responsibilities:**
 - Apply statistical theories to solve real-world problems.
 - Design experiments and surveys.
 - Interpret data using statistical techniques.
- **Skills:** R, Python, SPSS, SAS, advanced statistics and probability.

8. AI Researcher / Data Science Researcher

- **Responsibilities:**
 - Innovate and develop new algorithms and models.
 - Publish findings in academic journals or internal reports.
 - Focus on cutting-edge techniques like deep learning, reinforcement learning, etc.
 - **Skills:** Mathematics, deep learning, research methodology, scientific writing.
-

1.4 Life cycle of Data Science

The life cycle of data science refers to the structured process followed to extract meaningful insights from data and deliver actionable results. It consists of a series of steps that help ensure a successful data science project. Here is a breakdown of the typical Data Science Life Cycle:

1. Problem Definition

- Understand and define the business or research problem.
- Clarify objectives, deliverables, and success criteria.
- Stakeholder involvement is crucial.

 **Example:** A company wants to predict customer churn.

2. Data Collection

- Gather relevant data from different sources (databases, APIs, web scraping, sensors, etc.).
- Types: structured (tables), semi-structured (JSON, XML), unstructured (text, images).

 **Example:** Extract customer data, transactions, complaints, etc.

3. Data Cleaning (Data Preparation)

- Handle missing values, outliers, and inconsistencies.
- Convert formats, normalize values, and remove duplicates.
- Prepare data for analysis.

 **Tools:** Pandas, NumPy, OpenRefine.

4. Exploratory Data Analysis (EDA)

- Use statistics and visualizations to understand data.
- Identify patterns, trends, correlations, and anomalies.

🔗 **Tools:** Matplotlib, Seaborn, Plotly, Power BI, Tableau.

5. Feature Engineering

- Select and transform variables to improve model performance.
- Encode categorical data, scale features, create new variables.

🔗 **Example:** Converting age into age groups.

6. Model Building

- Choose suitable algorithms (Regression, Classification, Clustering, etc.).
- Train the model on the dataset.
- Split data into training/testing sets (often 70:30 or 80:20).

🔗 **Libraries:** scikit-learn, TensorFlow, PyTorch, XGBoost.

7. Model Evaluation

- Measure performance using appropriate metrics:
 - Classification: Accuracy, Precision, Recall, F1-score

- Regression: RMSE, MAE, R^2
- Perform cross-validation.

8. Model Tuning

- Optimize hyperparameters to improve model performance.
- Use techniques like Grid Search or Random Search.

9. Deployment

- Integrate the model into production (web app, API, dashboard).
- Monitor model performance in real-world scenarios.

 **Tools:** Flask, FastAPI, Docker, AWS, Azure, Streamlit.

10. Monitoring & Maintenance

- Continuously track model accuracy and relevance.
- Update model as new data becomes available (model drift).

Unit 3: Programming for Data Science

3.1 What is NumPy

NumPy (Numerical Python) is a powerful open-source Python library used for numerical and scientific computing. It provides support for large, multi-dimensional arrays and matrices, along with a collection of mathematical functions to operate on these arrays.

3.2 Differences between Python lists and NumPy arrays

Feature	Python List	NumPy Array
Data Type	Can hold different data types	Homogeneous (all elements of same type)
Performance	Slower for numerical operations	Much faster due to optimized C backend
Memory Usage	Higher memory usage	Lower memory usage
Functionality	Limited mathematical functions	Rich set of mathematical operations
Broadcasting	Not supported	Supported (operations auto-apply across elements)

Feature	Python List	NumPy Array
Multidimensional Support	Indirect (list of lists)	Built-in support for multidimensional arrays
Element-wise Operations	Need loops or list comprehension	Directly supported (e.g., array1 + array2)
Library Requirement	Built-in to Python	Requires importing NumPy (import numpy as np)
Indexing/Slicing	Basic	Advanced slicing, masking, and boolean indexing
Use Case	General-purpose	Scientific computing, data analysis, machine learning
E.g	A=[2,4,5]	import numpy as np a = np.array([1, 2, 3])

3.3 NumPy Arrays

From Python Lists/Tuples

```
import numpy as np
```

```
arr1 = np.array([1, 2, 3])      # 1D array
```

```
arr2 = np.array([[1, 2], [3, 4]])  # 2D array
```

```
arr3 = np.array(((1, 2), (3, 4)))  # From tuple of tuples
```

Creating Arrays with Built-in Functions

1. np.zeros() – Array of all zeros

`np.zeros(5)` # 1D array

`np.zeros((2, 3))` # 2D array

2. np.ones() – Array of all ones

`np.ones(5)`

`np.ones((2, 3))`

3. np.full() – Array filled with a constant value

`np.full((2, 3), 7)`

4. np.empty() – Empty (uninitialized) array (Creates an array without initializing the entries. The values in the array can be random and depend on the state of the memory.)

`np.empty((2, 2))`

Ranges and Sequences

1. np.arange():

Generates an array with regularly incrementing values, similar to Python's built-in `range()` but returns an array

`np.arange(0, 10, 2)` # [0, 2, 4, 6, 8]

2. **np.linspace():**

Returns evenly spaced numbers over a specified interval.

```
np.linspace(0, 1, 5)  # [0. , 0.25, 0.5 , 0.75, 1. ]
```

3. **reshape()**: It is used to change the shape of an existing array without changing its data.

Syntax

```
numpy.reshape(a, newshape)
```

OR

```
a.reshape(newshape)
```

E.g

```
a = np.array([1, 2, 3, 4, 5, 6])
```

```
reshaped = a.reshape((2, 3))
```

```
print(reshaped)
```

NumPy Properties

1.dtype (Data Type): Describes the type of elements in the array (e.g., int32, float64, bool, etc.).

```
arr = np.array([1, 2, 3])
```

```
print(arr.dtype)  # Output: int64 (or int32 depending on system)
```

2.shape: Returns a tuple representing the dimensions of the array.

```
arr = np.array([[1, 2], [3, 4], [5, 6]])
```

```
print(arr.shape) # Output: (3, 2)
```

3.ndim (Number of Dimensions):Returns the number of axes (dimensions) of the array.

```
arr = np.array([[1, 2], [3, 4]])
```

```
print(arr.ndim) # Output: 2
```

4.size:Total number of elements in the array.

```
arr = np.array([[1, 2], [3, 4]])
```

```
print(arr.size) # Output: 4
```

5. itemsize:Size (in bytes) of each element in the array.

```
arr = np.array([1, 2, 3], dtype=np.int32)
```

```
print(arr.itemsize) # Output: 4 (bytes per int32 element)
```

Indexing in NumPy

1D Array Indexing

```
arr = np.array([10, 20, 30, 40])
```

```
print(arr[0]) # Output: 10
```

```
print(arr[-1]) # Output: 40 (last element)
```

2D Array Indexing

```
arr = np.array([[1, 2, 3], [4, 5, 6]])
```

```
print(arr[0][1]) # Output: 2
```

```
print(arr[1, 2]) # Output: 6 (preferred syntax)
```

3D Array Indexing

```
arr = np.array([[[1,2],[3,4]], [[5,6],[7,8]]])
```

```
print(arr[0, 1, 1]) # Output: 4
```

Slicing in NumPy

1D Array Slicing

```
arr = np.array([10, 20, 30, 40, 50])  
  
print(arr[1:4])    # Output: [20 30 40]  
  
print(arr[:3])     # Output: [10 20 30]  
  
print(arr[::-2])   # Output: [10 30 50]
```

2D Array Slicing

```
arr = np.array([[1,2,3],[4,5,6],[7,8,9]])  
  
print(arr[0:2, 1:3]) # Output: [[2 3], [5 6]]  
  
print(arr[:, 1])    # Output: [2 5 8] (all rows, column 1)
```

Boolean Indexing

```
arr = np.array([1, 2, 3, 4, 5])  
  
print(arr[arr > 3]) # Output: [4 5]
```

Fancy Indexing

```
arr = np.array([10, 20, 30, 40, 50])  
  
print(arr[[0, 2, 4]]) # Output: [10 30 50]
```

Iterating numPy**1.Iterating Over 1D Arrays**

```
import numpy as np  
  
arr = np.array([10, 20, 30, 40])  
  
for x in arr:
```

```
print(x)
```

2. Iterating Over 2D Arrays (Row by Row)

```
arr2d = np.array([[1, 2], [3, 4], [5, 6]])
```

```
for row in arr2d:
```

```
    print(row)
```

o/p

```
[1 2]
```

```
[3 4]
```

```
[5 6]
```

3. Iterating Over Each Element (Nested Loops)

```
for row in arr2d:
```

```
    for item in row:
```

```
        print(item)
```

4. Using np.nditer() (Efficient Flat Iteration)

```
for x in np.nditer(arr2d):
```

```
    print(x)
```

Array Operations in NumPy

1. Arithmetic Operations (Element-wise)

```
a = np.array([10, 20, 30])
```

```
b = np.array([1, 2, 3])
```

```
print(a + b) # [11 22 33]
```

```
print(a - b) # [9 18 27]
```

```
print(a * b) # [10 40 90]
```

```
print(a / b) # [10. 10. 10.]
```

2. Scalar Operations

```
print(a + 5) # [15 25 35]
```

```
print(a * 2) # [20 40 60]
```

3. Aggregation Operations

```
arr = np.array([[1, 2], [3, 4]])
```

```
print(arr.sum()) # 10
```

```
print(arr.mean()) # 2.5
```

```
print(arr.max()) # 4
```

```
print(arr.min()) # 1
```

```
print(arr.std()) # Standard deviation
```

```
print(arr.sum(axis=0)) # [4 6] → column-wise
```

```
print(arr.sum(axis=1)) # [3 7] → row-wise
```

argmin() and argmax() in NumPy

```
arr = np.array([10, 50, 30, 5, 80])
```

```
print(np.argmin(arr)) # 3 → index of 5 (min value)
```

```
print(np.argmax(arr)) # 4 → index of 80 (max value)
```

4. Broadcasting

```
a = np.array([[1], [2], [3]]) # Shape (3, 1)
```

```
b = np.array([10, 20]) # Shape (2,)
```

```
print(a + b)
```

5.Comparison Operations in NumPy

1.Element-wise Comparison with Scalars

```
a = np.array([10, 20, 30, 40])
```

```
print(a > 25) # [False False True True]
```

```
print(a == 20) # [False True False False]
```

```
print(a != 10) # [False True True True]
```

2. Element-wise Comparison Between Arrays

```
b = np.array([10, 15, 30, 35])
```

```
print(a > b) # [False True False True]
```

```
print(a == b) # [ True False True False]
```

3. Masking / Filtering Using Comparison

```
1.print(a[a > 25]) # [30 40]
```

2.

```
matrix = np.array([[1, 2], [3, 4]])
```

```
print(matrix > 2)
```

o/p

```
[[False False]
```

```
 [ True  True]]
```

4. Useful Functions with Comparison

Function	Description
np.all()	True if all values are True
np.any()	True if any value is True
np.where(condition)	Returns indices or values where condition is True
np.count_nonzero()	Counts number of True elements

```
print(np.any(a > 25))    # True
print(np.all(a > 5))     # True
print(np.where(a > 25))  # (array([2, 3]),)
print(np.count_nonzero(a > 25)) # 2
```

6.Array Manipulation in NumPy

1. Reshaping Arrays

```
a = np.array([1, 2, 3, 4, 5, 6])
b = a.reshape((2, 3))
print(b)
# [[1 2 3]
#  [4 5 6]]
```

Use -1 to let NumPy automatically compute the dimension:

```
a.reshape((-1, 2)) # Automatically infers rows
```

2. Flattening Arrays

```
a = np.array([[1, 2], [3, 4]])
```

```
flat = a.ravel()

print(flat) # [1 2 3 4]

flat_copy = a.flatten()

print(flat_copy)
```

4. Concatenation

Function	Description
<code>concatenate()</code>	Join along an existing axis
<code>vstack()</code>	Stack vertically (row-wise)
<code>hstack()</code>	Stack horizontally (column-wise)
<code>stack()</code>	Stack with new axis added

1. `np.concatenate()`

Syntax

```
np.concatenate((array1, array2), axis=0)
```

E.g 1D Arrays

```
a = np.array([1, 2])

b = np.array([3, 4])

print(np.concatenate((a, b)))

# Output: [1 2 3 4]
```

E.g 2D Arrays (Row-wise)

```
a = np.array([[1, 2]])  
b = np.array([[3, 4]])  
print(np.concatenate((a, b), axis=0))
```

Output:

```
# [[1 2]
```

```
# [3 4]]
```

2D Arrays (Column-wise)

```
a = np.array([[1, 2], [3, 4]])  
b = np.array([[5, 6], [7, 8]])  
print(np.concatenate((a, b), axis=1))
```

Output:

```
# [[1 2 5 6]
```

```
# [3 4 7 8]]
```

2. np.vstack() – Vertical Stack (Row-wise)

```
a = np.array([1, 2])  
b = np.array([3, 4])  
  
print(np.vstack((a, b)))
```

Output:

```
# [[1 2]
```

```
# [3 4]]
```

3. np.hstack() – Horizontal Stack (Column-wise)

```
a = np.array([[1], [2]])
```

```
b = np.array([[3], [4]])
```

```
print(np.hstack((a, b)))
```

Output:

```
# [[1 3]
```

```
# [2 4]]
```

4. **np.stack()** – Add a new axis while joining

```
a = np.array([1, 2])
```

```
b = np.array([3, 4])
```

```
print(np.stack((a, b), axis=0))
```

Output:

```
# [[1 2]
```

```
# [3 4]]
```

```
print(np.stack((a, b), axis=1))
```

Output:

```
# [[1 3]
```

```
# [2 4]]
```

Inserting & Deleting element in numPy

1.Inserting Elements with np.insert()

Syntax

```
np.insert(array, index, values, axis=None)
```

E.g

```
arr = np.array([10, 20, 30])  
new_arr = np.insert(arr, 1, 15) # Insert 15 at index 1  
print(new_arr)  
# Output: [10 15 20 30]
```

Example 2: Insert in 2D array along axis

```
arr2d = np.array([[1, 2], [3, 4]])  
new_arr2d = np.insert(arr2d, 1, [9, 9], axis=0) # Insert row at index 1  
print(new_arr2d)  
# Output:  
# [[1 2]  
# [9 9]  
# [3 4]]
```

2. Deleting Elements with np.delete()

Syntax

```
np.delete(array, index, axis=None)
```

E.g

```
arr = np.array([10, 20, 30, 40])  
new_arr = np.delete(arr, 2) # Delete element at index 2  
print(new_arr)  
# Output: [10 20 40]
```

Example 2: Delete from 2D array along axis

```
arr2d = np.array([[1, 2], [3, 4], [5, 6]])  
new_arr2d = np.delete(arr2d, 1, axis=0) # Delete row at index 1  
print(new_arr2d)
```

Output:

```
# [[1 2]  
# [5 6]]
```

Notes:

- These operations do not modify the original array; they return a new array.
- Inserting or deleting elements is not very efficient for large arrays (NumPy is optimized for fixed-size arrays).

Random Arrays

1.np.random.random():Generates an array with random floats in the range [0.0, 1.0).

1D array

```
arr1 = np.random.random(5)  
print("1D array:", arr1)
```

2D array

```
arr2 = np.random.random((2, 3))  
print("2D array:\n", arr2)
```

2. np.random.rand(): generates random floats in [0.0, 1.0), but uses arguments directly as dimensions (no tuple required).

```
arr = np.random.rand(3, 4) # 3x4 array  
print(arr)
```

3. np.random.randint(): Generates random integers in a specified range.

```
arr = np.random.randint(10, size=(3, 3)) # Integers from 0 to 9  
print(arr)  
  
arr2 = np.random.randint(1, 100, size=(2, 4)) # Integers from 1 to 99  
print(arr2)
```

4. np.random.randn(): Samples from a standard normal distribution (mean 0, variance 1).

```
arr = np.random.randn(3, 3) # 3x3 array  
print(arr)
```

5. np.random.uniform(): Generates random floats from a uniform distribution over a specified range.

```
arr = np.random.uniform(low=10, high=50, size=(2, 3))  
print(arr)
```

6. numpy.random.choice(): Used to randomly select elements from an array (with or without replacement).

Syntax

```
np.random.choice(a, size=None, replace=True, p=None)
```

Where

Parameters:

- **a:** 1-D array or int (if int, it treats as `np.arange(a)`)
- **size:** Number of elements to choose
- **replace:** True (default) for sampling with replacement, False for without
- **p:** Probabilities for each element

1. Randomly select 3 elements from a list

```
arr = [10, 20, 30, 40, 50]
result = np.random.choice(arr, size=3)
print(result)
# Output: [30 10 20] (random output)
```

2. Without replacement

```
np.random.choice(arr, size=3, replace=False)
```

3. With probability

```
np.random.choice(arr, size=2, p=[0.1, 0.2, 0.3, 0.2, 0.2])
```

7.numpy.random.shuffle():Used to shuffle the original array in-place (does not return a new array).

Syntax

```
np.random.shuffle(arr)
```

Only shuffles the first axis (1D arrays get fully shuffled; 2D arrays shuffle rows only).

1. Shuffle a 1D array

```
arr = np.array([1, 2, 3, 4, 5])
np.random.shuffle(arr)
print(arr)
```

```
# Output: [3 5 2 1 4] (random order)
```

2. Shuffle rows of a 2D array

```
arr2d = np.array([[1, 2], [3, 4], [5, 6]])
```

```
np.random.shuffle(arr2d)
```

```
print(arr2d)
```

```
# Output:
```

```
# [[5 6]
```

```
# [1 2]
```

```
# [3 4]] (rows shuffled)
```

Difference between choice() & shuffle()

Feature	np.random.choice()	np.random.shuffle()
Return	New array	Shuffles in-place
Replacement	Optional (replace=True/False)	Not applicable
Axis Affected	Not axis-based	First axis only
Use Case	Random sampling	Random rearrangement

8. np.random.seed(): Setting a random seed ensures reproducibility of random numbers — you get the same random output each time you run the code

Syntax

```
np.random.seed(seed_value)
```

Where

seed_value: An integer (like 0, 1, 42, etc.)

With setting a seed:

```
np.random.seed(42)
```

```
print(np.random.randint(1, 100))
```

Output: same every time (e.g., 52)

Without setting a seed:

```
print(np.random.randint(1, 100))
```

Output: different every time

Mathematical and Statistical Functions**1. Mathematical Functions**

Function	Description	Example
<code>np.add(x, y)</code>	Element-wise addition	<code>np.add([1, 2], [3, 4]) → [4, 6]</code>
<code>np.subtract(x, y)</code>	Element-wise subtraction	<code>[1, 2] - [3, 4] → [-2, -2]</code>
<code>np.multiply(x, y)</code>	Element-wise multiplication	<code>[2, 3] * [4, 5] → [8, 15]</code>
<code>np.divide(x, y)</code>	Element-wise division	<code>[4, 9] / [2, 3] → [2, 3]</code>
<code>np.power(x, y)</code>	Exponentiation	<code>np.power([2, 3], 2) → [4, 9]</code>
<code>np.sqrt(x)</code>	Square root	<code>np.sqrt([4, 9]) → [2, 3]</code>
<code>np.exp(x)</code>	Exponential	<code>np.exp(1) → 2.718...</code>
<code>np.log(x)</code>	Natural log	<code>np.log([1, np.e])</code>

Function	Description	Example
<code>np.sin(x)</code>	Trigonometric sine	<code>np.sin(np.pi/2) → 1.0</code>
<code>np.round(x)</code>	Round to nearest integer	<code>np.round(3.14159, 2) → 3.14</code>

2. Statistical Functions

Function	Description	Example
<code>np.mean(x)</code>	Arithmetic mean	<code>np.mean([1, 2, 3]) → 2.0</code>
<code>np.median(x)</code>	Median value	<code>np.median([1, 2, 3]) → 2.0</code>
<code>np.average(x, weights=...)</code>	Weighted average	<code>np.average([1,2,3], weights=[1,1,2])</code>

Function	Description	Example
<code>np.std(x)</code>	Standard deviation	<code>np.std([1, 2, 3])</code>
<code>np.var(x)</code>	Variance	<code>np.var([1, 2, 3])</code>
<code>np.ptp(x)</code>	Peak-to-peak (max - min)	<code>np.ptp([1, 2, 8]) → 7</code>
Function	Description	Example
<code>np.min(x)</code>	Minimum value	<code>np.min([4, 7, 1]) → 1</code>
<code>np.max(x)</code>	Maximum value	<code>np.max([4, 7, 1]) → 7</code>
<code>np.argmin(x)</code>	Index of min value	<code>np.argmin([4, 7, 1]) → 2</code>
<code>np.argmax(x)</code>	Index of max value	<code>np.argmax([4, 7, 1]) → 1</code>
<code>np.percentile(x, q)</code>	Percentile value	<code>np.percentile([1, 2, 3], 50) → 2</code>

Linear Algebra with NumPy

Operation	Function/Method
Dot product	<code>np.dot(A, B)</code> or <code>A @ B</code>
Matrix transpose	<code>A.T</code>
Identity matrix	<code>np.eye(n)</code>
Determinant	<code>np.linalg.det(A)</code>
Inverse	<code>np.linalg.inv(A)</code>
Rank	<code>np.linalg.matrix_rank(A)</code>
Eigenvalues/vectors	<code>np.linalg.eig(A)</code>
Solve $Ax = b$	<code>np.linalg.solve(A, b)</code>

1. Dot Product / Matrix Multiplication

1D vectors

```
a = np.array([1, 2])
```

```
b = np.array([3, 4])
```

```
np.dot(a, b)      # Output: 11 (1×3 + 2×4)
```

```
a @ b            # Same as dot product in Python 3.5+
```

-

2D matrices

```
A = np.array([[1, 2], [3, 4]])
```

```
B = np.array([[5, 6], [7, 8]])
```

```
np.dot(A, B)
```

```
# OR
```

```
np.matmul(A, B)
```

2. Transpose of a Matrix

```
A.T # Transpose of matrix A
```

3. Identity Matrix

```
np.eye(3) # 3x3 Identity matrix
```

4. Determinant

```
np.linalg.det(A)
```

5. Inverse of a Matrix

```
np.linalg.inv(A) # Only if A is square and non-singular
```

6. Rank of a Matrix

```
np.linalg.matrix_rank(A)
```

7. Eigenvalues and Eigenvectors

```
eig_vals, eig_vecs = np.linalg.eig(A)
```

8. Solving Linear Equations: $Ax = b$

Solve equations like $2x + 3y = 8$ and $x + 2y = 5$:

```
A = np.array([[2, 3], [1, 2]])
```

```
b = np.array([8, 5])
```

```
x = np.linalg.solve(A, b)
```

```
print(x) # Solution [x, y]
```

Introduction to Pandas

1.1 What is Pandas

Pandas is open-source Python library which is used for data manipulation and analysis. It consists of data structures and functions to perform efficient operations on data. It is well-suited for working with tabular data such as spreadsheets or SQL tables. It is used in data science because it works well with other important libraries. It is built on top of the NumPy library as it makes easier to manipulate and analyze.

1.2 Importance of Pandas in Data Analysis

1. Easy Data Handling with DataFrames

- Pandas introduces two powerful data structures: Series (1D) and DataFrame (2D).
- These allow for tabular data manipulation similar to Excel or SQL, making it intuitive for analysts.

2. Data Cleaning & Preprocessing

- Handling missing data (NaN values), duplicate removal, type conversion, and filtering are made simple and fast.
- Functions like `.dropna()`, `.fillna()`, `.astype()` streamline the preparation of clean data.

3. Data Exploration and Analysis

- Offers powerful statistical functions like `.mean()`, `.median()`, `.describe()`, `.value_counts()` for quick summaries.
- Helps discover trends, anomalies, and distributions early in the analysis process.

4. Data Manipulation

- Easy to perform operations like:
 - Merging/joining datasets: `.merge()`, `.join()`
 - Grouping and aggregation: `.groupby()`
 - Reshaping: `.pivot()`, `.melt()`
 - Sorting and indexing

5. Time Series Analysis

- Pandas has built-in support for time series data with tools for:
 - Date parsing, frequency conversion
 - Rolling windows, shifting, and resampling

6. Integration with Other Tools

- Works seamlessly with NumPy, Matplotlib, Scikit-learn, and SQL databases.
- Compatible with Jupyter notebooks, enhancing interactive data exploration.

7. Performance

- Optimized for performance, especially for medium-sized datasets.
- Built on top of NumPy, it ensures fast computation and efficient memory use.

1.3 Data structures in Pandas(Series, DataFrame)

2 types Data structures in Pandas(Series, DataFrame)

1.Series in Pandas

2. DataFrame in Pandas

1.Series in Pandas

A Series is a one-dimensional labeled array capable of holding any data type—integers, strings, floats, Python objects, etc. It is the basic building block of a Pandas DataFrame.

Key Features of Series

Feature	Description
One-dimensional	Stores data in a single column-like structure
Labeled index	Each value has an associated label (index)
Homogeneous type	Values are usually of the same data type
Fast & efficient	Built on top of NumPy for performance
Can be sliced/indexed	Supports powerful indexing & selection

E.g 1

```
import pandas as pd
```

```
s = pd.Series([10, 20, 30, 40])
```

```
print(s)
```

o/p

```
0    10
```

```
1    20
```

```
2    30
```

```
3    40
```

```
dtype: int64
```

Eg. 2

```
s = pd.Series([100, 200, 300], index=['a', 'b', 'c'])
```

```
print(s)
```

o/p

```
a    100
```

```
b    200
```

```
c    300
```

```
dtype: int64
```

1.Series function

1. head() and tail()

Returns the first/last n elements of the Series.

```
s = pd.Series([10, 20, 30, 40, 50])
```

```
print(s.head(3)) # First 3 elements
```

```
print(s.tail(2)) # Last 2 elements
```

2. dtype, shape, size

Gives the type, shape, and size.

```
print(s.dtype) # int64
```

```
print(s.shape) # (5,)
```

```
print(s.size)    # 5
```

3. index and values

```
print(s.index)   # RangeIndex(start=0, stop=5, step=1)
```

```
print(s.values)  # array([10, 20, 30, 40, 50])
```

4. sum(), mean(), max(), min()

```
print(s.sum())   # 150
```

```
print(s.mean())  # 30.0
```

```
print(s.max())   # 50
```

```
print(s.min())   # 10
```

5. describe()

Provides a statistical summary.

```
print(s.describe())
```

o/p

```
count    5.000000
```

```
mean     30.000000
```

```
std      15.811388
```

```
min      10.000000
```

```
25%      20.000000
```

```
50%      30.000000
```

```
75%      40.000000
```

```
max      50.000000
```

```
dtype: float64
```

6. value_counts()

Counts frequency of each unique value.

```
s2 = pd.Series([10, 20, 10, 30, 20, 10])  
print(s2.value_counts())
```

o/p

10 3

20 2

30 1

7. unique()

Returns an array of unique values.

```
print(s2.unique()) # [10 20 30]
```

8. sort_values() and sort_index()

Sort by value or index.

```
print(s.sort_values()) # Sort by values  
print(s.sort_index()) # Sort by index (default)
```

9. apply()

Apply a function to each element.

```
print(s.apply(lambda x: x * 2)) # Double each element
```

10. map()

Similar to apply(), used for element-wise transformations.

```
print(s.map({10: 'Ten', 20: 'Twenty', 30: 'Thirty'}))
```

11. isin()

Checks if elements are in the given list.

```
print(s.isin([20, 40])) # Returns True/False for matching values
```

12. between()

Checks if values fall within a range.

```
print(s.between(20, 40)) # True for values between 20 and 40
```

13. drop_duplicates() and duplicated()

```
print(s2.drop_duplicates()) # Keeps unique values
```

```
print(s2.duplicated()) # True for repeated values
```

2. DataFrame in Pandas

A DataFrame is the most important and widely used data structure in the Pandas library. It represents 2-dimensional, tabular data (like an Excel spreadsheet or SQL table), where data is organized in rows and columns.

Feature	Description
2D structure	Data is arranged in rows and columns.
Heterogeneous data	Columns can hold different data types (int, float, string, etc.).
Labeled axes	Rows and columns have labels (index and column names).

Feature	Description
Powerful operations	Filtering, grouping, merging, reshaping, aggregation, etc.
Integration	Works seamlessly with other Python libraries like NumPy, Matplotlib, and Scikit-learn.

1. From a dictionary of lists

```
import pandas as pd
```

```
data = {  
    'Name': ['Alice', 'Bob', 'Charlie'],  
    'Age': [24, 30, 35],  
    'City': ['Pune', 'Mumbai', 'Delhi']  
}
```

```
df = pd.DataFrame(data)
```

```
print(df)
```

o/p

```
   Name  Age  City  
0  Alice   24  Pune  
1   Bob   30 Mumbai  
2 Charlie   35  Delhi
```

2. From a list of dictionaries

```
data = [  
    {'Name': 'Alice', 'Age': 24, 'City': 'Pune'},
```

```
{'Name': 'Bob', 'Age': 30, 'City': 'Mumbai'}  
]
```

```
df = pd.DataFrame(data)  
  
print(df)
```

3. Reading Data

```
df = pd.read_csv('data.csv') //From CSV  
  
df = pd.read_excel('data.xlsx', sheet_name='Sheet1') //From Excel  
  
df = pd.read_json('data.json') // From JSON  
  
import sqlite3 //From SQL  
  
conn = sqlite3.connect('mydatabase.db')  
  
df = pd.read_sql_query('SELECT * FROM tablename', conn)
```

4. Writing Data

```
df.to_csv('output.csv', index=False) //To CSV  
  
df.to_excel('output.xlsx', index=False) //To Excel  
  
df.to_json('output.json', orient='records', lines=True) //To JSON  
  
df.to_sql('tablename', conn, if_exists='replace', index=False) //To SQL
```

5. head() – View Top Rows

```
df.head()  
  
df.head(10) # First 10 rows
```

Note: Shows the first 5 rows by default.

6. tail() – View Bottom Rows

```
df.tail()
```

```
df.tail(10) # First 10 rows
```

Note: Shows the last 5 rows by default.

7.info() – Summary of DataFrame

Gives a concise summary including:

- Number of non-null values
- Data types of each column
- Memory usage

```
df.info()
```

8. describe() – Statistical Summary

Gives summary statistics (count, mean, std, min, max, etc.) for numerical columns.

```
df.describe()
```

```
df.describe(include='all') //To include non-numeric columns
```

9. shape – Dimensions

Returns a tuple: (rows, columns)

```
df.shape
```

10. dtypes – Data Types

Shows the data type of each column.

```
df.dtypes
```

```
import pandas as pd
```

```
data = {'Name': ['Alice', 'Bob', 'Charlie'], 'Age': [25, 30, 35], 'Salary': [50000, 60000, 70000]}
```

```
df = pd.DataFrame(data)
```

```
print(df.head())
```

```
print(df.tail())
```

```
print(df.info())
```

```
print(df.describe())
```

```
print(df.shape)
```

```
print(df.dtypes)
```

11.loc[] vs .iloc[] in Pandas

Feature	.loc[]	.iloc[]
Access by	Label/index name	Integer position
Syntax	df.loc[row_label, column_label]	df.iloc[row_index, column_index]
Includes	End index included (label-based slicing)	End index excluded (like Python)
Usable with	Strings, booleans, slices	Integer positions only
	<pre>import pandas as pd data = { 'Name': ['Alice', 'Bob', 'Charlie'], 'Age': [25, 30, 35], 'City': ['Pune', 'Mumbai', 'Delhi'] } df = pd.DataFrame(data, index=['a', 'b', 'c']) print(df)</pre>	

Feature	.loc[]	.iloc[]
	df.loc['a'] # Row with label 'a' df.loc['a', 'Name'] # 'Alice' df.loc[['a', 'b'], ['Name', 'City']] # Subset of rows and columns df.iloc[0] # First row df.iloc[0, 0] # 'Alice' df.iloc[0:2, 0:2] # First 2 rows and first 2 columns (Note: exclusive)	

12. slicing

import pandas as pd

```
data = {'Name': ['Amit', 'Neha', 'Raj', 'Simran'],
```

```
       'Marks': [85, 90, 75, 95]}
```

```
df = pd.DataFrame(data)
```

```
# Slice rows 1 to 3 (excluding row 3)
```

```
print(df[1:3])
```

o/p

```
Name Marks
```

```
1 Neha    90
```

```
2 Raj     75
```

2. Slicing using .loc[] (label-based)

```
# Slice rows using labels (row index must be labeled properly)
```

```
df = pd.DataFrame(data, index=['a', 'b', 'c', 'd'])
```

```
print(df.loc['b':'d']) # Includes 'd'
```

o/p

Name Marks

b Neha 90

c Raj 75

d Simran 95

3. Slicing using .iloc[] (integer position-based)

Slice rows and columns by integer position

```
print(df.iloc[1:3, 0:2]) # Rows 1 to 2, Columns 0 to 1
```

o/p

Name Marks

b Neha 90

c Raj 75

4.Slicing Columns

Select only the 'Name' column

```
print(df['Name']) # Single column
```

Select multiple columns

```
print(df[['Name', 'Marks']])
```

13.Boolean Indexing in DataFrame (Pandas)

```
import pandas as pd
```

```
data = {'Name': ['Amit', 'Neha', 'Raj', 'Simran'],  
        'Marks': [85, 90, 75, 95],  
        'Subject': ['Math', 'Science', 'Math', 'Science']}
```

```
df = pd.DataFrame(data)
```

```
print(df)
```

o/p

	Name	Marks	Subject
0	Amit	85	Math
1	Neha	90	Science
2	Raj	75	Math
3	Simran	95	Science

1. Single Condition

Select rows where Marks > 80

```
print(df[df['Marks'] > 80])
```

o/p

	Name	Marks	Subject
0	Amit	85	Math
1	Neha	90	Science
3	Simran	95	Science

2. Multiple Conditions

Marks > 80 and Subject is 'Science'

```
print(df[(df['Marks'] > 80) & (df['Subject'] == 'Science')])
```

Use & for AND, | for OR, and parentheses around conditions.

3. Using .isin() for Multiple Matches

Select students whose subject is either Math or English-

```
print(df[df['Subject'].isin(['Math', 'English'])])
```

4. Negating a Condition

Select students who did not take Math

```
print(df[~(df['Subject'] == 'Math')])
```

3.Data Cleaning

Function	Purpose
isnull()	Detect missing (NaN) values
notnull()	Detect non-missing values
dropna()	Remove missing values
fillna()	Replace missing values with something

1. isnull()

Returns True for missing (NaN) values and False otherwise.

```
import pandas as pd
```

```
import numpy as np
```

```
data = pd.Series([10, np.nan, 30, None, 50])
```

```
print(data.isnull())
```

o/p

0 False

1 True

2 False

3 True

4 False

dtype: bool

2. notnull()

Opposite of isnull(). Returns True for non-missing values.

```
print(data.notnull())
```

o/p

0 True

1 False

2 True

3 False

4 True

dtype: bool

3. dropna()

Removes missing (NaN) values.

```
print(data.dropna())
```

o/p

0 10.0

2 30.0

```
4  50.0
```

```
dtype: float64
```

4. fillna()

Replaces NaN with a specified value.

```
print(data.fillna(0)) # Replace NaN with 0
```

o/p

```
0  10.0
```

```
1   0.0
```

```
2  30.0
```

```
3   0.0
```

```
4  50.0
```

```
dtype: float64
```

Replacing Values in Pandas

In Pandas, replacing values is commonly done using the `.replace()` method. It allows you to substitute values in a Series or DataFrame with specified values.

Syntax

```
DataFrame.replace(to_replace, value, inplace=False)
```

Where

- `to_replace`: The value(s) you want to replace.
- `value`: The new value(s).

- inplace: If True, modifies the data in place (no return); otherwise, returns a new object.

E.g

1. Replace Single Value in Series

```
s = pd.Series([1, 2, 3, 4, 2])
```

```
s.replace(2, 99)
```

Output:

```
[1, 99, 3, 4, 99]
```

2. Replace Multiple Values in Series

```
s.replace([1, 2], [100, 200])
```

Output:

```
[100, 200, 3, 4, 200]
```

3. Replace Values in DataFrame

```
df = pd.DataFrame({
```

```
    'A': [1, 2, 3],
```

```
    'B': [4, 2, 5]
```

```
})
```

```
df.replace(2, 99)
```

	A	B
0	1	4
1	99	99
2	3	5

Renaming Columns or Index in Pandas

In Pandas, renaming column names or index labels is commonly done using the `.rename()` method or by directly assigning new names.

Syntax

```
DataFrame.rename(mapper=None, columns=None, index=None, inplace=False)
```

Where

columns: Dictionary to rename columns.

index: Dictionary to rename row indexes.

inplace: If True, makes changes in place.

Eg.

1. Rename Columns Using `.rename()`

```
df = pd.DataFrame({  
    'A': [1, 2],  
    'B': [3, 4]  
})  
df.rename(columns={'A': 'Alpha', 'B': 'Beta'})
```

O/p

	Alpha	Beta
0	1	3
1	2	4

2. Rename Index Labels

```
df.rename(index={0: 'First', 1: 'Second'})
```

o/p

	A	B
First	1	3
Second	2	4

Duplicate

Function	Use
<code>df.duplicated()</code>	Find duplicate rows
<code>df.drop_duplicates()</code>	Remove duplicate rows

1. DataFrame.duplicated()

This function returns a Boolean Series indicating whether each row is a duplicate.

Syntax

```
DataFrame.duplicated(subset=None, keep='first')
```

Where

subset: Column(s) to check for duplicates.

keep:

- 'first': Mark duplicates except for the first occurrence.
- 'last': Mark duplicates except for the last occurrence.
- False: Mark all duplicates as True.

E.g

```
df = pd.DataFrame({  
    'Name': ['Ram', 'Sham', 'Ram', 'Geeta'],  
    'Score': [90, 85, 90, 88]  
})
```

```
df.duplicated()
```

o/p

0 False

1 False

2 True # Duplicate of row 0

3 False

2. DataFrame.drop_duplicates(): This removes duplicate rows.

Syntax

```
DataFrame.drop_duplicates(subset=None, keep='first', inplace=False)
```

Where

- subset: Columns to consider.
- keep: 'first', 'last', or False.
- inplace: Modify DataFrame directly.

E.g

```
df.drop_duplicates()
```

o/p

Removes the 2nd "Ram" entry.

3. Duplicates in Specific Columns

Syntax

```
df.duplicated(subset='Name')
```

```
df.drop_duplicates(subset='Name', keep='last')
```

4. Remove All Duplicates (No Keeps)

```
df[~df.duplicated(keep=False)]
```

o/p

Keeps only rows that occur once.

Type Conversion in Pandas

In Pandas, you can convert the data types of columns using methods like `.astype()` and `pd.to_numeric()`. This is often necessary for cleaning or preparing data for analysis or modeling.

1. Using `.astype()`

Syntax

```
DataFrame['column'] = DataFrame['column'].astype(new_type)
```

E.g

```
df = pd.DataFrame({'A': ['1', '2', '3'], 'B': [4.0, 5.2, 6.1]})
```

```
# Convert column A from string to int
```

```
df['A'] = df['A'].astype(int)
```

```
# Convert column B to string
```

```
df['B'] = df['B'].astype(str)
```

2. Using `pd.to_numeric()`

Useful for converting strings to numbers, especially when there may be invalid values.

E.g

```
df = pd.DataFrame({'A': ['1', '2', 'three']})
```

```
df['A'] = pd.to_numeric(df['A'], errors='coerce')
```

o/p

```
0    1.0
```

```
1    2.0
```

```
2   NaN
```

- **errors='coerce':** Replaces invalid parsing with NaN.
- **errors='ignore':** Leaves invalid entries unchanged.

3. Using `pd.to_datetime()`

Converts strings to datetime objects

```
df['date'] = pd.to_datetime(df['date'])
```

Note:

Convert Multiple Columns at Once

```
df = df.astype({'A': int, 'B': float})
```

Sorting in Pandas

1. Sorting by Column Values

Syntax

```
df.sort_values(by='column_name', ascending=True, inplace=False)
```

Eg

```
df.sort_values(by='Score') # ascending order by score
```

```
df.sort_values(by='Score', ascending=False) # descending order
```

Multiple Columns:

```
df.sort_values(by=['Score', 'Name'], ascending=[False, True])
```

2. Sorting by Index

Syntax

```
df.sort_index(axis=0, ascending=True, inplace=False)
```

Where

- axis=0: sort by row index
- axis=1: sort by column names
- ascending=True: sort in ascending order

Eg

```
data = {'Name': ['Alice', 'Bob', 'Charlie'],
```

```
       'Score': [85, 95, 90]}
```

```
df = pd.DataFrame(data, index=['b', 'a', 'c'])
```

```
print(df.sort_index()) # sort by row index
```

3. rank()

In Pandas, the .rank() function is used to provide rankings of elements in a Series or DataFrame.

Ranking means assigning a numerical rank to each element, where the lowest value typically gets the rank 1 (by default).

Syntax

```
DataFrame.rank(axis=0, method='average', numeric_only=None, na_option='keep',
ascending=True)
```

Methods of Ranking:

Parameter	Description
axis	0 for index (columns), 1 for columns (rows)
method	{'average', 'min', 'max', 'first', 'dense'} – method to assign ranks
na_option	{'keep', 'top', 'bottom'} – how to rank NaN values
ascending	If True, lower values get lower ranks (rank 1 is the smallest)
Method	Description
average	Default. Average of ranks if there's a tie
min	Minimum rank in case of ties
max	Maximum rank in case of ties
first	Ranks assigned in order they appear
dense	Like 'min', but ranks increase by 1 (no gaps)

Eg

Sample data

```
data = pd.Series([20, 10, 30, 20])
```

Default ranking (method='average')

```
print(data.rank())
```

o/p

```
0    2.5
```

```
1    1.0
```

```
2    4.0
```

```
3    2.5
```

```
dtype: float64
```

Using different methods:

```
print("min:\n", data.rank(method='min'))
```

```
print("max:\n", data.rank(method='max'))
```

```
print("dense:\n", data.rank(method='dense'))
```

```
print("first:\n", data.rank(method='first'))
```

o/p

```
min:
```

```
0    2.0
```

```
1    1.0
```

2 4.0

3 2.0

max:

0 3.0

1 1.0

2 4.0

3 3.0

dense:

0 2.0

1 1.0

2 3.0

3 2.0

first:

0 2.0

1 1.0

2 4.0

3 3.0

DataFrame Ranking Example:

```
df = pd.DataFrame({  
    'Math': [88, 92, 95, 70],
```

```
'Science': [80, 85, 90, 70]
})
```

```
# Rank students row-wise
```

```
print(df.rank(axis=1))
```

o/p

```
Math Science
0  2.0    1.0
1  2.0    1.0
2  2.0    1.0
3  1.5    1.5
```

Common String Methods in Pandas

Method	Description
str.lower()	Converts to lowercase
str.upper()	Converts to uppercase
str.title()	Converts to title case
str.strip()	Removes leading/trailing whitespace
str.lstrip() / str.rstrip()	Removes whitespace from left/right
str.replace(pat, repl)	Replaces substring
str.contains(pat)	Returns True/False if pattern is found
str.startswith() / str.endswith()	Checks if string starts/ends with a pattern

Method	Description
str.len()	Returns string length
str.split(sep)	Splits string by separator
str.cat(sep)	Concatenates strings in Series
str.extract(pattern)	Extracts regex match
str.findall(pattern)	Returns all regex matches
str.get(i)	Gets ith element from each string
str.pad(width)	Pads strings to a specified width
str.zfill(width)	Pads numeric string with zeros
str.isdigit() / str.isalpha()	Checks for numeric/alphabetic

```
import pandas as pd
```

```
data = pd.Series([' apple ', 'Banana', 'cherry', '123'])
```

```
# Convert to lowercase
```

```
print(data.str.lower())
```

```
# Remove whitespace
```

```
print(data.str.strip())
```

```
# Replace  
print(data.str.replace('a', '@'))
```

```
# Check if it contains 'a'  
print(data.str.contains('a'))
```

```
# Length of each string  
print(data.str.len())
```

```
# Extract digits using regex  
print(data.str.extract('(\d+)'))
```

o/p

0 apple

1 banana

2 cherry

3 123

dtype: object

0 apple

1 Banana

2 cherry

3 123

dtype: object

0 @pple
1 B@n@n@
2 cherry
3 123

dtype: object

0 True
1 True
2 False
3 False

dtype: bool

0 7
1 6
2 6
3 3

dtype: int64

0

0 NaN
1 NaN

2 NaN

3 123

Date/Time Operations in Pandas

Function/Attribute	Description
pd.to_datetime()	Converts string or object to datetime
.dt accessor	Access datetime properties like year, month, etc.
pd.date_range()	Creates a sequence of dates
pd.Timedelta	Represents a duration or difference between two dates
pd.DatetimeIndex	Index with datetime values for time-series data

Eg.

1. pd.to_datetime()

```
data = pd.Series(['2023-01-01', '2023-06-15', '2024-03-10'])
```

```
dates = pd.to_datetime(data)
```

```
print(dates)
```

Eg.

2. .dt accessor

```
print(dates.dt.year)    # Year
```

```
print(dates.dt.month)   # Month
```

```
print(dates.dt.day)     # Day
```

```
print(dates.dt.weekday) # Weekday (Monday=0)
```

```
print(dates.dt.day_name()) # Day name
```

3..Date Arithmetic

Add days

```
print(dates + pd.Timedelta(days=10))
```

```
# Difference between dates
```

```
print(dates[2] - dates[0]) # Output: Timedelta
```

Grouping and Aggregation in Pandas

Grouping and aggregation in pandas are used to analyze and summarize data based on certain keys or categories — just like SQL GROUP BY.

Syntax

DataFrame.groupby(by)[column].agg(function)

Where

- by: column(s) to group by
- agg(): aggregation function(s) like sum, mean, count, etc.

Common Aggregation Functions

Function	Description
sum()	Sum of values
mean()	Mean (average)
count()	Count non-null entries

Function	Description
min() / max()	Minimum / maximum value
median()	Median value
std()	Standard deviation
agg()	Apply multiple functions

Example 1: Simple Grouping and Aggregation

```
import pandas as pd
```

```
# Sample Data
```

```
df = pd.DataFrame({  
    'Department': ['Sales', 'Sales', 'HR', 'HR', 'IT'],  
    'Employee': ['Alice', 'Bob', 'Charlie', 'David', 'Eva'],  
    'Salary': [50000, 60000, 45000, 47000, 70000]  
})
```

```
# Group by Department and get average salary
```

```
result = df.groupby('Department')['Salary'].mean()
```

```
print(result)
```

o/p

Department

HR 46000.0

IT 70000.0

Sales 55000.0

Name: Salary, dtype: float64

Example 2: Multiple Aggregations

Sum and Count salary by department

```
result = df.groupby('Department')['Salary'].agg(['sum', 'count', 'mean'])
```

```
print(result)
```

o/p

	sum	count	mean
Department			
HR	92000	2	46000.0
IT	70000	1	70000.0
Sales	110000	2	55000.0

Example 3: Group by Multiple Columns

```
df['Gender'] = ['F', 'M', 'M', 'M', 'F']
```

Group by Department and Gender

```
result = df.groupby(['Department', 'Gender'])['Salary'].mean()
```

```
print(result)
```

o/p

Department	Gender	
HR	M	46000.0
IT	F	70000.0
Sales	F	50000.0
	M	60000.0

Name: Salary, dtype: float64

Example 4: Using Custom Function

```
# Custom aggregation function using lambda  
result = df.groupby('Department')['Salary'].agg(lambda x: x.max() - x.min())  
print(result)
```

o/p

Department

HR 2000

IT 0

Sales 10000

Name: Salary, dtype: int64

Merging, Joining, and Concatenation**1. merge():** Used for: Combining DataFrames on common columns or indices**Syntax**

```
pd.merge(left, right, how='inner', on=None, left_on=None, right_on=None)
```

Parameter

Parameter	Description
left, right	DataFrames to be merged
How	Type of join: 'inner', 'outer', 'left', 'right'
On	Column name(s) to join on (must be present in both)
left_on, right_on	Column names to join on in left and right DataFrames

Parameter	Description
left_index, right_index	Join on index (if set to True)

Types of Joins in merge()

Join Type	Description
inner	Only matching rows (default)
outer	All rows from both, with NaN where no match
left	All rows from left + matching from right
right	All rows from right + matching from left

E.g

Example 1: merge() on a common column

```
df1 = pd.DataFrame({  
    'ID': [1, 2, 3],  
    'Name': ['Alice', 'Bob', 'Charlie']  
})
```

```
df2 = pd.DataFrame({  
    'ID': [2, 3, 4],
```

```
'Marks': [85, 90, 95]
})
result = pd.merge(df1, df2, on='ID', how='inner')
print(result)
```

o/p

	ID	Name	Marks
0	2	Bob	85
1	3	Charlie	90

Example 2: left join

```
result = pd.merge(df1, df2, on='ID', how='left')
print(result)
```

o/p

	ID	Name	Marks
0	1	Alice	NaN
1	2	Bob	85.0
2	3	Charlie	90.0

Example 3: right join

```
result = pd.merge(df1, df2, on='ID', how='right')
print(result)
```

o/p

	ID	Name	Marks
0	2	Bob	85
1	3	Charlie	90

```
2 4 NaN 95
```

Example 4: outer join

```
result = pd.merge(df1, df2, on='ID', how='outer')
```

```
print(result)
```

o/p

	ID	Name	Marks
0	1	Alice	NaN
1	2	Bob	85.0
2	3	Charlie	90.0
3	4	NaN	95.0

Example 5: Merge on different column names

```
df3 = pd.DataFrame({
```

```
    'emp_id': [2, 3, 4],
```

```
    'Marks': [85, 90, 95]
```

```
})
```

```
result = pd.merge(df1, df3, left_on='ID', right_on='emp_id', how='inner')
```

```
print(result)
```

o/p

	ID	Name	emp_id	Marks
0	2	Bob	2	85
1	3	Charlie	3	90

2.join()

The .join() function in pandas is used to combine two DataFrames by index (or by a key column). It is often simpler and faster than merge() when working with indices.

Syntax

DataFrame.join(other, on=None, how='left', lsuffix="", rsuffix="", sort=False)

Parameter	Description
other	The DataFrame to join with
on	Column to join on (if not using index)
how	Type of join: 'left' (default), 'right', 'outer', 'inner'
lsuffix/rsuffix	Suffix to add if there are overlapping column names

Eg

```
df1 = pd.DataFrame({'Name': ['Alice', 'Bob', 'Charlie']}, index=[1, 2, 3])
```

```
df2 = pd.DataFrame({'Marks': [85, 90, 95]}, index=[2, 3, 4])
```

```
result = df1.join(df2, how='inner')
```

```
print(result)
```

o/p

Name Marks

2 Bob 85

3 Charlie 90

Difference between merge() & join()

Feature	join()	merge()
Type	Method (df1.join(df2))	Function (pd.merge(df1, df2))
Default Key	Index (of caller & other DataFrame)	Common column(s), must specify with on if needed
Join on Columns	Only if specified using on	Yes, flexible with on, left_on, right_on
Join on Index	Yes (default behavior)	Yes, using left_index=True, right_index=True
Flexibility	Less flexible	More powerful and customizable
Use Case	Simple joins based on index	Complex joins like SQL: inner, outer, left, right
Join Types	'left' (default), 'right', 'inner', 'outer'	'inner' (default), 'left', 'right', 'outer'
Handling Overlaps	Needs lsuffix and rsuffix if column names overlap	Automatically handles or can use suffixes

3. concat()

The `concat()` function in pandas is used to concatenate (combine) multiple pandas objects along a particular axis (rows or columns).

Syntax

```
pd.concat(objs, axis=0, join='outer', ignore_index=False, keys=None)
```

Parameter	Description
objs	A list/tuple of DataFrames or Series
axis	0 for row-wise, 1 for column-wise (default is 0)
join	'outer' (default): union of indexes; 'inner': intersection
ignore_index	If True, ignores existing index and creates a new one
keys	Add hierarchical index (useful in multi-level concat)

Example 1: Row-wise Concatenation (default axis=0)

```
df1 = pd.DataFrame({'A': ['A0', 'A1'], 'B': ['B0', 'B1']})
```

```
df2 = pd.DataFrame({'A': ['A2', 'A3'], 'B': ['B2', 'B3']})
```

```
result = pd.concat([df1, df2])
```

```
print(result)
```

o/p

A B

```
0 A0 B0
```

```
1 A1 B1
```

```
0 A2 B2
```

```
1 A3 B3
```

Example 2: Reset Index After Concatenation

```
pd.concat([df1, df2], ignore_index=True)
```

Example 3: Column-wise Concatenation (axis=1)

```
pd.concat([df1, df2], axis=1)
```

Example 4: Inner Join (only common columns)

```
pd.concat([df1, df2], join='inner')
```

Example 5: Adding Keys (MultiIndex)

```
pd.concat([df1, df2], keys=['Group1', 'Group2'])
```

concat() vs merge() vs join()

Feature	concat()	merge()	join()
Purpose	Combine along rows/columns	Combine like SQL joins	Join DataFrames using index or column
Default Axis	axis=0 → row-wise	N/A (uses key column)	axis=1 (column-wise join)
Join Key	Not required	Required (on, left_on, right_on)	Defaults to index, can use on

Feature	concat()	merge()	join()
Join Type	outer by default	inner by default (left, right, outer)	left by default
Flexibility	Moderate	Most flexible	Limited to simpler joins
Handles Duplicates?	May need ignore_index=True	Automatically manages duplicates	May need lsuffix/rsuffix if overlapping
Use Case	Stacking DFs, combining without keys	SQL-style join on columns or index	Quick joins on index or single column
MultiIndex Support	Yes (via keys)	Yes	Yes
Syntax Style	Function: pd.concat()	Function: pd.merge()	Method: df1.join()

Visualization with Pandas

Matplotlib

Matplotlib is a data visualization library in Python. It is widely used to create static, interactive, and animated plots and charts. It provides tools to generate high-quality graphs in a variety of formats and styles.

1.plot(): The plot() function is used to create **line plots**, which are one of the most basic types of plots in Matplotlib.

Syntax

matplotlib.pyplot.plot(x, y, format_string, **kwargs)

- x: X-axis data (list, array, or series)
- y: Y-axis data
- format_string (optional): Specifies line style, color, and marker
- **kwargs: Additional arguments to customize the plot

E.g

```
import matplotlib.pyplot as plt
```

```
x = [1, 2, 3, 4]
```

```
y = [2, 4, 1, 3]
```

```
plt.plot(x, y)
```

```
plt.title("Simple Line Plot")
```

```
plt.xlabel("X-axis")
```

```
plt.ylabel("Y-axis")
```

```
plt.show()
```

E.g

Multiple Plots in One Graph

x = [1, 2, 3, 4]

y1 = [1, 4, 9, 16]

y2 = [2, 3, 5, 7]

```
plt.plot(x, y1, label="y = x^2")
```

```
plt.plot(x, y2, label="y = 2x-1")
```

```
plt.legend()
```

```
plt.show()
```

E.g

```
import matplotlib.pyplot as plt
```

```
import numpy as np
```

```
xpoints = np.array([1, 8])
```

```
ypoints = np.array([3, 10])
```

```
plt.plot(xpoints, ypoints, marker = 'o')
```

```
plt.plot(xpoints, ypoints, linestyle = 'dotted')
```

```
linestyle = 'dashed'
```

```
plt.show()
```

Label & Title

```
plt.title("Sports Watch Data")
```

```
plt.xlabel("Average Pulse")
```

```
plt.ylabel("Calorie Burnage")
```

```
plt.ylim(10,200) //y max scale
```

```
plt.xlim(10,300) //x max scale
```

2.grid(): Grid lines help improve **readability** by making it easier to interpret the data points on the graph. You can add grid lines using the `plt.grid()` function.

Syntax

```
plt.grid(True) # Show grid lines
```

Eg.

```
import matplotlib.pyplot as plt
```

```
x = [1, 2, 3, 4]
```

```
y = [10, 20, 15, 25]
```

```
plt.plot(x, y)
```

```
plt.title("Line Plot with Grid")
```

```
plt.xlabel("X-axis")
```

```
plt.ylabel("Y-axis")
```

```
plt.grid(True) # Enable grid
```

```
plt.show()
```

3.subplot(): The `subplot()` function is used to **create multiple plots** (subplots) in a **single figure** useful for comparing different datasets or visualizations side by side.

Syntax

```
plt.subplot(nrows, ncols, index)
```

- `nrows`: Number of rows
- `ncols`: Number of columns
- `index`: Position of the subplot (starts from 1, not 0)

E.g

```
import matplotlib.pyplot as plt
# Create 1 row, 2 columns of subplots
plt.subplot(1, 2, 1) # First subplot
plt.plot([1, 2, 3], [4, 5, 6])
plt.title("Plot 1")

plt.subplot(1, 2, 2) # Second subplot
plt.plot([1, 2, 3], [6, 5, 4])
plt.title("Plot 2")

plt.tight_layout() # Avoid overlap
plt.show()
```

4. **scatter()**: It is used to create **scatter plots**, which are ideal for **visualizing the relationship between two numerical variables**.

Syntax

```
plt.scatter(x, y, **kwargs)
```

- x: X-axis values
- y: Y-axis values
- **kwargs: Optional parameters to customize appearance (e.g. color, size, marker)

E.g

```
import matplotlib.pyplot as plt
x = [5, 7, 8, 7, 2, 17, 2, 9]
y = [99, 86, 87, 88, 100, 86, 103, 87]

plt.scatter(x, y)
```

```
plt.title("Simple Scatter Plot")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.grid(True)
plt.show()
```

5. bar(): This function is used to create **vertical bar charts**, which are ideal for comparing **categorical data** using rectangular bars.

Syntax

```
plt.bar(x, height, **kwargs)
```

- x: Categories or positions on the X-axis
- height: Values (heights) of the bars
- **kwargs: Optional parameters (color, width, align, etc.)

E.g

```
x = ['Math', 'Science', 'English', 'History']
scores = [85, 90, 75, 88]
plt.bar(x, scores)
plt.title("Student Scores")
plt.xlabel("Subjects")
plt.ylabel("Marks")
plt.show()
```

6.pie(): The pie() function is used to create pie charts, which display data as a circular chart divided into wedges — useful for showing proportional data.

Syntax

```
plt.pie(data, labels=labels, **kwargs)
```

- data: Values for each slice
- labels: Category names for each slice
- **kwargs: Optional settings (colors, explode, autopct, etc.)

E.g

```
labels = ['Math', 'Science', 'English', 'History']
```

```
scores = [85, 90, 75, 88]
```

```
plt.pie(scores, labels=labels)
```

```
plt.title("Subject Score Distribution")
```

```
plt.show()
```

7.hist(): A histogram is a type of bar chart that shows the frequency distribution of a dataset — perfect for understanding how data is spread (e.g., normal, skewed).

Syntax

```
plt.hist(data, bins=number, **kwargs)
```

- data: List or array of values
- bins: Number of intervals (default is 10)
- **kwargs: Optional settings like color, edgecolor, orientation, etc.

E.g

```
ages = [18, 22, 20, 21, 25, 30, 24, 22, 26, 28, 27, 19, 23, 20]
```

```
plt.hist(ages)
```

```
plt.title("Age Distribution")
```

```
plt.xlabel("Age")
```

```
plt.ylabel("Frequency")
```

```
plt.show()
```

Seaborn

Seaborn is a data visualization library built on top of Matplotlib. It provides a high-level interface for creating attractive and informative statistical graphics — especially useful when working with Pandas DataFrames.

- Easier and cleaner syntax than Matplotlib
- Beautiful **default styles** and themes
- Powerful functions for **statistical plots**
- Works seamlessly with **Pandas** data structures
- Automatic **aggregation**, **coloring**, and **categorical separation**

1. Distribution Plots

Plot Type	Function	Description
Histogram	sns.histplot()	Distribution of a single variable
KDE Plot	sns.kdeplot()	Kernel Density Estimate (smoothed curve)
ECDF Plot	sns.ecdfplot()	Empirical cumulative distribution

2. Categorical Plots

Plot Type	Function	Description
Bar Plot	sns.barplot()	Shows mean/estimation with error bars
Count Plot	sns.countplot()	Counts the number of observations
Box Plot	sns.boxplot()	Shows distribution with quartiles
Violin Plot	sns.violinplot()	Combines boxplot and KDE
Strip Plot	sns.stripplot()	Individual data points (jittered)

Plot Type	Function	Description
Swarm Plot	sns.swarmplot()	Non-overlapping points (like bees)
Point Plot	sns.pointplot()	Line plot for categorical data
Cat Plot	sns.catplot()	Multi-purpose for any categorical plot

3. Relational Plots

Plot Type	Function	Description
Scatter Plot	sns.scatterplot()	Relationship between two numeric variables
Line Plot	sns.lineplot()	Trends over continuous or time data
Rel Plot	sns.relplot()	Combines scatter and line with facets

4. Matrix Plots

Plot Type	Function	Description
Heatmap	sns.heatmap()	Colored matrix (e.g., correlations)
Clustermap	sns.clustermap()	Heatmap with hierarchical clustering

1. pointplot(): It is used to **plot point estimates and confidence intervals** for **categorical** data. It shows the **mean (or other estimator)** of a quantitative variable and provides **error bars** to represent uncertainty (e.g., standard deviation or confidence interval)

Syntax

```
sns.pointplot(x=None, y=None, hue=None, data=None, estimator=<function mean>, ci=95)
```

Parameter	Description
x, y	Names of variables in data to be plotted on the x/y axis
hue	Used for grouping data by a third variable
data	Dataset (Pandas DataFrame)
estimator	Function to compute the estimate (default is mean)
ci	Size of confidence interval (e.g., 95 for 95% CI), or "sd" for standard deviation
markers	Style of point markers
linestyles	Style of line (e.g., dashed, solid)

Eg

```
import seaborn as sns
```

```
import matplotlib.pyplot as plt
```

```
# Load sample dataset
```

```
tips = sns.load_dataset("tips")
```

```
# Point plot: average total_bill by day
```

```
sns.pointplot(x="day", y="total_bill", data=tips)
```

```
plt.title("Average Total Bill by Day")
```

```
plt.show()
```

2. **scatterplot()**: The **scatterplot()** function in Seaborn is used to draw a **scatter plot** — a plot of **two continuous variables** to show relationships or correlations. It supports grouping and visual enhancements like colors, markers, sizes, etc.

Syntax

`sns.scatterplot(x=None, y=None, hue=None, style=None, size=None, data=None)`

Parameter	Description
x, y	Variables to plot on the x and y axes
hue	Grouping variable (different colors)
style	Different markers based on category
size	Vary marker sizes
data	DataFrame (Pandas)
palette	Color palette
s	Marker size (if not using size)

```
import seaborn as sns
```

```
import matplotlib.pyplot as plt
```

```
# Load example dataset
```

```
tips = sns.load_dataset("tips")
```

```
# Basic scatter plot
```

```
sns.scatterplot(x="total_bill", y="tip", data=tips)
```

```
plt.title("Total Bill vs Tip")
```

```
plt.show()
```

3. **countplot()** : This function is used to **show the counts of observations** in each **categorical bin** using bars. It's a simple and effective way to **visualize the frequency** of different categories.

Syntax

```
sns.countplot(x=None, y=None, hue=None, data=None)
```

Parameter	Description
x, y	Categorical variable (one required)
hue	For subgrouping (another categorical variable)
data	DataFrame containing the variables
palette	Set color palette
order	Custom order of categories
orient	Vertical or horizontal bar

E.g

```
import seaborn as sns
```

```
import matplotlib.pyplot as plt
```

```
# Load example dataset
```

```
tips = sns.load_dataset("tips")
```

```
# Basic count plot of days
```

```
sns.countplot(x="day", data=tips)
```

```
plt.title("Number of Customers per Day")
```

```
plt.show()
```

4. **barplot()**: The **barplot()** function in Seaborn is used to **plot the mean (or other estimator) of a numerical variable** across categories. It's different from `countplot()` — while `countplot()` shows **counts**, `barplot()` shows **aggregated values** (like mean, median, etc.).

Syntax

```
sns.barplot(x=None, y=None, hue=None, data=None, estimator=<function mean>, ci=95)
```

Parameter	Description
x, y	Categorical and numerical variables
hue	Subgrouping variable
data	Pandas DataFrame
estimator	Function to apply (e.g., mean, sum)
ci	Confidence interval size (e.g., 95), or "sd" for standard deviation
palette	Color palette for bars
capsize	Adds caps on error bars
orient	Vertical or horizontal bars

E.g

```
import seaborn as sns
```

```
import matplotlib.pyplot as plt
```

```
# Load dataset
```

```
tips = sns.load_dataset("tips")
```

```
# Bar plot: average tip by day
sns.barplot(x="day", y="tip", data=tips)

plt.title("Average Tip by Day")
plt.show()
```

5. **heatmap()** :A **heatmap** is a **2D graphical representation** of data where individual values are represented as colors. In Seaborn, `sns.heatmap()` is typically used to visualize:

- **Correlation matrices**
- **Pivot tables**
- **Confusion matrices**
- **Matrix-style datasets**

Syntax

```
sns.heatmap(data, annot=False, fmt='.2g', cmap=None, linewidths=0, linecolor='white')
```

Parameter	Description
Data	2D data (like DataFrame, NumPy array)
Annot	Show the values inside cells (True/False)
Fmt	String formatting for annot values
Cmap	Color map (e.g., "coolwarm", "viridis", "Blues")
Linewidths	Width of lines between cells
Linecolor	Color of cell lines
vmin, vmax	Color scale limits

E.g

```
import seaborn as sns
import matplotlib.pyplot as plt
# Load dataset
```

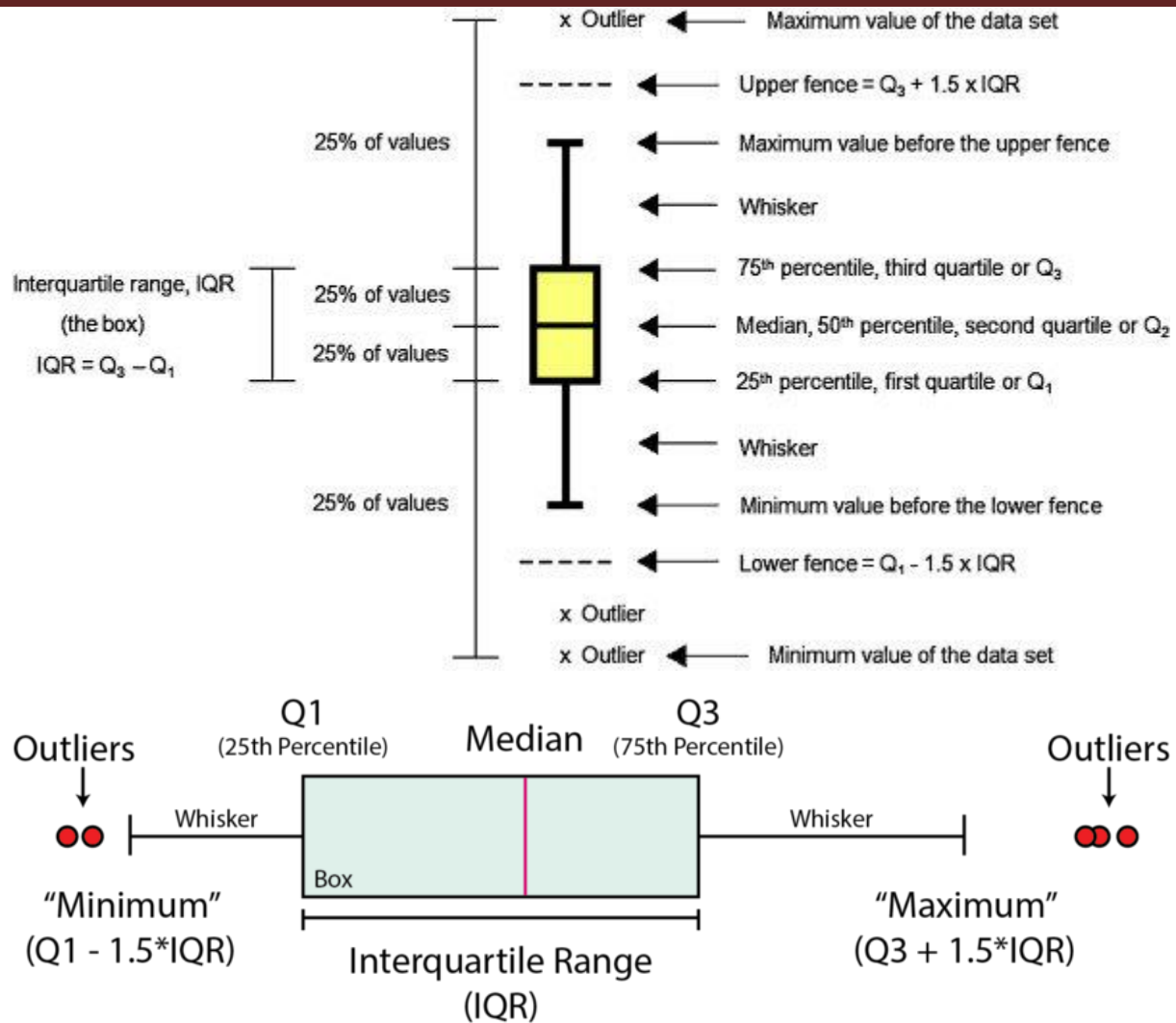
```
df= sns.load_dataset("tips")  
sns.heatmap(df.corr(),annot = True)
```

6. **boxplot()**:The **boxplot()** function in Seaborn is used to draw a **box-and-whisker plot**, which displays the **distribution, central tendency, and variability** of a dataset. It's great for detecting **outliers** and **comparing distributions** across categories.

Syntax

```
sns.boxplot(x=None, y=None, hue=None, data=None, orient=None)
```

Parameter	Description
x, y	Categorical and numerical variables
Hue	For subgrouping
Data	DataFrame (Pandas)
palette	Color palette
orient	"v" for vertical, "h" for horizontal
width	Width of each box



E.g

```
import seaborn as sns
```

```
import matplotlib.pyplot as plt
```

```
# Load dataset
```

```
tips = sns.load_dataset("tips")
```

```
# Basic boxplot: Tip by day
```

```
sns.boxplot(x="day", y="tip", data=tips)
```

```
plt.title("Boxplot of Tips by Day")
```

```
plt.show()
```

Comparison: Matplotlib vs Seaborn

Feature	Matplotlib	Seaborn
Library Type	Low-level plotting library	High-level statistical visualization library (built on Matplotlib)
Syntax	More detailed, requires more code	Concise and simpler syntax
Integration with Pandas	Requires manual handling (e.g. <code>df['col']</code>)	Natively supports Pandas DataFrames
Default Style	Basic/default styling	Attractive and modern default styles
Customization	Highly customizable (fine control)	Limited to common plot types, but easy to style
Statistical Plots	Needs manual calculation (mean, CI, etc.)	Built-in support for statistical visualization
Types of Plots	All basic plots (line, bar, pie, scatter, etc.)	Line, bar, box, violin, KDE, heatmap, pairplot, etc.
Themes	Manual styling	Predefined themes: darkgrid, whitegrid, ticks, etc.
Learning Curve	Steeper	Easier for beginners
Flexibility	Very flexible, low-level	Less flexible, but great for quick insights
Best For	Custom & technical plots	Statistical analysis, exploratory data analysis (EDA)

Unit 4: Data Wrangling and Preprocessing & Exploratory Data Analysis (EDA)

1.Data Wrangling and Preprocessing

Data collection is the first step in a data science project. It involves gathering data from various sources, typically in the following ways:

1. APIs (Application Programming Interfaces)

Definition:

APIs allow you to programmatically access data provided by online services (e.g., Twitter, Google Maps, OpenWeather, etc.).

Common Tools/Libraries:

- requests (Python)
- http.client, urllib
- json (to parse API response)

```
import requests
```

```
url = "https://api.openweathermap.org/data/2.5/weather?q=Pune&appid=your_api_key"
```

```
response = requests.get(url)
```

```
data = response.json()
```

```
print(data['main']['temp'])
```

Advantages:

- Real-time data
- Structured and reliable
- Legal and stable access

2. Web Scraping

Definition:

Extracting data from websites that don't offer APIs, by parsing HTML pages.

Common Tools/Libraries:

- BeautifulSoup (for parsing HTML)
- requests (to fetch pages)
- Selenium (for dynamic JavaScript content)

- Scrapy (advanced framework)

E.g

```
from bs4 import BeautifulSoup
import requests
url = "https://example.com"
response = requests.get(url)
soup = BeautifulSoup(response.text, "html.parser")
titles = soup.find_all("h2")
for title in titles:
    print(title.text)
```

Advantages:

- Access to almost any public data
- No dependency on APIs

Caution:

- Follow website terms of use
- Use robots.txt to check permissions

3. Databases**Definition:**

Collecting data stored in structured databases using SQL queries or connectors.

Types of Databases:

- **Relational:** MySQL, PostgreSQL, SQLite
- **NoSQL:** MongoDB, Cassandra

Common Tools/Libraries:

- sqlite3 or mysql.connector (Python)
- SQLAlchemy (ORM)
- pymongo (MongoDB)

```
import sqlite3
conn = sqlite3.connect('example.db')
cursor = conn.cursor()
cursor.execute("SELECT * FROM students")
rows = cursor.fetchall()
for row in rows:
    print(row)
conn.close()
```

Advantages:

- Structured and reliable
- Supports large-scale data storage
- Easily integrated into pipelines

2. Handling Missing Data

Missing data can negatively affect the accuracy of your model and analysis. It's essential to identify and deal with it properly.

Step 1: Detecting Missing Data

```
import pandas as pd
df = pd.read_csv("data.csv")
# Detect missing values
print(df.isnull().sum()) # Total null values per column
print(df.isnull().any()) # True/False if any null
```

1. Deletion Methods

Remove missing data if it's minimal or irrelevant.

a. Row-wise Deletion

Removes rows with **any** or **all** missing values.

```
df_cleaned = df.dropna()          # Drop rows with any missing values
df_cleaned = df.dropna(how='all')  # Drop rows where all values are missing
```

b. Column-wise Deletion

```
df_cleaned = df.dropna(axis=1)    # Drop columns with any missing values
```

2. Imputation Methods

Filling missing values using calculated or assumed data.

a. Constant Imputation

```
df['column'].fillna(0, inplace=True)    # Replace with 0
df['column'].fillna("Unknown", inplace=True)  # Replace with string
```

b. Mean/Median/Mode Imputation

```
df['age'].fillna(df['age'].mean(), inplace=True)    # Numerical
df['income'].fillna(df['income'].median(), inplace=True)
df['gender'].fillna(df['gender'].mode()[0], inplace=True) # Categorical
```

c. Forward/Backward Fill

```
df.fillna(method='ffill', inplace=True) # Forward fill
df.fillna(method='bfill', inplace=True) # Backward fill
```

d. Interpolate Values

Create a DataFrame with missing values

```
df = pd.DataFrame({'A': [1, 2, np.nan, 4, 5, np.nan, 7]})
```

```
# Using linear interpolation to fill missing values
```

```
df['A_linear'] = df['A'].interpolate()
# Using polynomial interpolation (order 2)
df['A_poly'] = df['A'].interpolate(method='polynomial', order=2)
print(df)
```

Method	Description	Use When
dropna()	Delete missing rows/columns	Few missing values, non-critical data
fillna()	Fill with constant/mean/median/mode	Common and simple cases
ffill() / bfill()	Fill based on previous/next value	Time series or sequential data
interpolate()	Fill using linear or polynomial interp	Numerical continuous data
ML-based	Predict missing values	Complex datasets, better accuracy required

3.Data transformation (scaling, normalization, encoding categorical variables)

Data transformation is the process of converting data into a suitable format or structure for analysis and modeling.

1.Scaling:Need/Benifit of scaling

Reason	Benefit
Different feature ranges	Prevents domination of larger scales
Gradient-based algorithms	Faster and smoother convergence
Distance-based algorithms	Accurate distance calculations
Regularization models	Balanced penalization
Visualization	Better data representation

Types of scaler

1. Scaling (Standardization)

Rescales features to have a mean of 0 and standard deviation of 1. Especially useful for models like SVM, KNN, Logistic Regression, etc.

Formula

$$z = \frac{x_i - \mu}{\sigma}$$

```
from sklearn.preprocessing import StandardScaler
```

```
*scaler = StandardScaler()
```

```
df_scaled = scaler.fit_transform(df[['feature1', 'feature2']])
```

2. Normalization (Min-Max Scaling)

Rescales values to a **fixed range**, usually [0, 1]. Useful when features have **different units/scales**.

Formula

$$X_scaled = (X - X_min) / (X_max - X_min)$$

E.g

```
from sklearn.preprocessing import MinMaxScaler
```

```
scaler = MinMaxScaler()
```

```
df_normalized = scaler.fit_transform(df[['feature1', 'feature2']])
```

3. MaxAbsScaler

- **What it does:** Scales each feature by its maximum absolute value.
- **Range:** [-1, 1]

- **Use case:** For data that is already centered at zero or sparse data.
- **Library:** from sklearn.preprocessing import MaxAbsScale

4. RobustScaler

- **What it does:** Uses the median and interquartile range (IQR) for scaling.
- **Formula:**

$$x' = \frac{x - \text{median}}{\text{IQR}}$$

- **Use case:** When data contains **outliers**.
- **Library:** from sklearn.preprocessing import RobustScaler

5. Normalizer

- **What it does:** Scales individual samples to have unit norm (e.g., L2 norm = 1).
- **Use case:** When you need to normalize samples (not features), such as in **text classification** or **cosine similarity** problems.
- **Library:** from sklearn.preprocessing import Normalizer

Scaler	Scales On	Handles Outliers	Range / Effect	Best For
StandardScaler	Mean & Std Dev	✗ No	Mean = 0, Std = 1	Normally distributed features
MinMaxScaler	Min & Max	✗ No	[0, 1] (default)	When features have different units
MaxAbsScaler	Max Abs Value	✗ No	[-1, 1]	Sparse data
RobustScaler	Median & IQR	☑ Yes	Based on robust statistics	Features with outliers
Normalizer	Sample-wise (rows)	✗ No	Each row scaled to norm = 1	Text data, distance-based ML

2.Types of Categorical Variables

Categorical variables are features that represent categories or groups, not numerical values. They are qualitative and can't be used directly in most machine learning models, so they must be encoded into numerical form.

Type	Description	Example
Nominal	Categories with no order or rank	Color: Red, Green, Blue
Ordinal	Categories with logical order	Size: Small, Medium, Large

Encoding Categorical Variables

1. Label Encoding

- Converts each category into a unique integer.
- Best for ordinal data.

```
from sklearn.preprocessing import LabelEncoder  
  
data = ['Small', 'Medium', 'Large']  
encoder = LabelEncoder()  
encoded = encoder.fit_transform(data)  
print(encoded) # Output: [2 1 0]
```

2. One-Hot Encoding

- Creates a new binary column for each category.
- Best for **nominal data**.

```
import pandas as pd  
  
df = pd.DataFrame({'Color': ['Red', 'Green', 'Blue']})  
encoded = pd.get_dummies(df, columns=['Color'])
```

```
print(encoded)
```

o/p

```
Color_Blue Color_Green Color_Red
0         0         0         1
1         0         1         0
2         1         0         0
```

3. Ordinal Encoding

- Converts categories into **ordered integers** (use only when order matters).

```
from sklearn.preprocessing import OrdinalEncoder
data = [['Small'], ['Medium'], ['Large']]
encoder = OrdinalEncoder(categories=[['Small', 'Medium', 'Large']])
encoded = encoder.fit_transform(data)
print(encoded) # Output: [[0.], [1.], [2.]]
```

4. Binary Encoding, Hash Encoding, Target Encoding (Advanced)

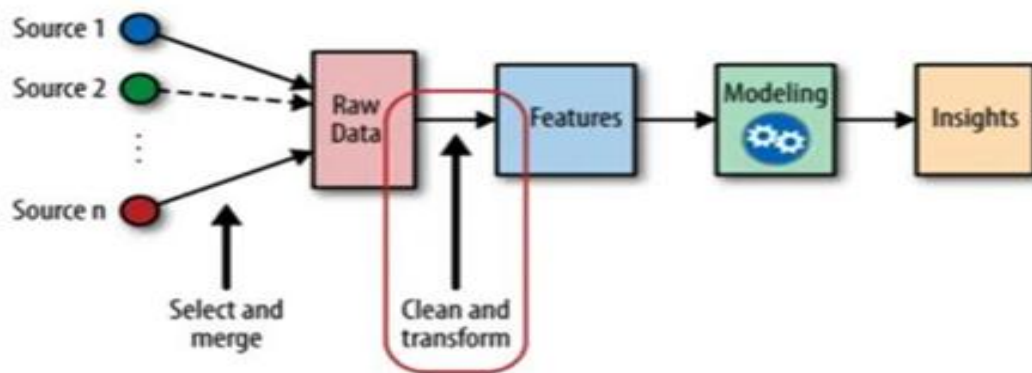
- Useful for **high-cardinality** features (many unique values).
- Available in libraries like `category_encoders`.

Encoding Type	Best For	Creates New Columns	Preserves Order
Label Encoding	Ordinal	No	Yes
One-Hot Encoding	Nominal	Yes	No
Ordinal Encoding	Ordinal	No	Yes
Binary/Hash/Target	High-cardinality	Depends	Depends

4.Feature Engineering

In Data Science, the performance of the model is depending on data preprocessing and data handling. Suppose if we build a model without Handling data, we got an accuracy of around 70%. By applying the Feature engineering on the same model there is a chance to increase the performance from 70% to more.

Simply, by using Feature Engineering we improve the performance of the model.



Feature Engineering Steps in Data Science

1. Understand the Data

Before engineering features, explore the dataset through:

- EDA (Exploratory Data Analysis)
- Understand variable types (categorical, numerical, datetime, text)
- Spot outliers, missing values, or skewed distributions

2. Handle Missing Data

Missing values can lead to model bias or errors.

Common Techniques:

- Imputation: Replace missing values using mean, median, mode, or more advanced methods (e.g., KNN, regression)

- Deletion: Remove rows/columns with too many missing values
- Indicator Variable: Add a binary flag if a value was missing

3. Encode Categorical Variables

Machine learning models require numerical input.

Encoding Type	Best For	Creates New Columns	Preserves Order
Label Encoding	Ordinal	No	Yes
One-Hot Encoding	Nominal	Yes	No
Ordinal Encoding	Ordinal	No	Yes
Binary/Hash/Target	High-cardinality	Depends	Depends

4. Feature Scaling (Normalization/Standardization)

Essential for distance-based or gradient-based algorithms.

Scaler	Scales On	Handles Outliers	Range / Effect	Best For
StandardScaler	Mean & Std Dev	✗ No	Mean = 0, Std = 1	Normally distributed features
MinMaxScaler	Min & Max	✗ No	[0, 1] (default)	When features have different units
MaxAbsScaler	Max Abs Value	✗ No	[-1, 1]	Sparse data
RobustScaler	Median & IQR	☑ Yes	Based on robust statistics	Features with outliers
Normalizer	Sample-wise (rows)	✗ No	Each row scaled to norm = 1	Text data, distance-based ML

5. Feature Transformation

Helps improve linearity or reduce skewness.

Feature Transformation is the process of applying mathematical or statistical operations to transform features into a format that improves **model performance, convergence, or interpretability**.

1. Log Transformation

Used to reduce **right-skewed** distributions and compress large values.

```
df['log_salary'] = np.log1p(df['salary']) # log(x + 1) to handle 0
```

2. Square Root / Cube Root Transformation

Also reduces skewness and stabilizes variance (less aggressive than log).

```
df['sqrt_income'] = np.sqrt(df['income'])
```

```
df['cbrt_income'] = np.cbrt(df['income'])
```

3. Power Transformation

General-purpose transformation for normalizing data.

Techniques:

- **Box-Cox Transformation:** For strictly positive values
- **Yeo-Johnson Transformation:** For positive and negative values

```
from sklearn.preprocessing import PowerTransformer
```

```
pt = PowerTransformer(method='yeo-johnson')
```

```
df[['transformed_col']] = pt.fit_transform(df[['original_col']])
```

6. Feature Creation (Feature Extraction)

Feature Creation is the process of generating new features based on domain knowledge or by observing patterns in the data. It is a form of feature engineering that can significantly improve the performance of a machine-learning model.

1. Datetime Feature Extraction

Extract useful parts from date or time fields.

```
df['year'] = df['date'].dt.year
```

```
df['month'] = df['date'].dt.month
```

```
df['day_of_week'] = df['date'].dt.dayofweek
```

```
df['is_weekend'] = df['day_of_week'].isin([5, 6])
```

2. Text-Based Feature Extraction

Convert text data into numerical features.

Examples:

- **Basic Features:**
 - Word count, character count
 - Average word length
 - Presence of specific keywords

```
df['word_count'] = df['text'].apply(lambda x: len(x.split()))
```

```
df['has_offer'] = df['text'].apply(lambda x: 'offer' in x.lower())
```

3. ⚙️ Ratio & Rate Features

Create features using **ratios** or **combinations** of existing numerical variables.

```
df['income_to_expense'] = df['income'] / (df['expenses'] + 1)
```

```
df['price_per_sqft'] = df['price'] / df['area']
```

4. Mathematical Transformations

Apply transformations to smooth, scale, or highlight relationships.

Examples:

- Logarithmic: $\log(x + 1)$
- Square, square root: $x**2$, `np.sqrt(x)`
- Binning: `pd.cut()` or `pd.qcut()`

5. Aggregations (Group-Based)

Useful for datasets with **ID-based or time-series data**.

Customer-level statistics

E.g

```
customer_avg = df.groupby('customer_id')['purchase_amount'].mean()
```

```
df['cust_avg_purchase'] = df['customer_id'].map(customer_avg)
```

6. Interaction Features

Combine two or more features to capture interactions.

E.g

```
df['area_income_interaction'] = df['area'] * df['income']
```

```
df['age_bucketed_income'] = df['income'] / (df['age'] + 1)
```

7. Domain Knowledge Features

Use **expert knowledge** to derive features meaningful to the problem.

Examples:

- In credit scoring: `debt_to_income_ratio`

- In health: $BMI = weight / (height ** 2)$
- In real estate: `distance_to_nearest_school`

8. Count/Frequency Encoding

Convert categorical columns to frequency-based features.

E,g

```
freq = df['city'].value_counts()
```

```
df['city_freq'] = df['city'].map(freq)
```

7. Binning/Bucketing

Convert continuous variables into discrete bins.

Examples:

- Age groups: (0–18), (19–35), (36–60), 60+
- Income brackets

8. Feature Interaction

Create new features by combining two or more variables.

Examples:

- $price_per_sqft = price / area$
- $income_to_debt_ratio = income / debt$

9. Dimensionality Reduction

Reduce number of features while retaining important information.

Techniques:

- PCA (Principal Component Analysis)
- t-SNE, UMAP (for visualization)
- Feature selection methods (e.g., Recursive Feature Elimination)

10. Feature Selection

Remove irrelevant or redundant features.

Methods:

1. Correlation analysis

Feature selection using **correlation analysis** helps identify **highly correlated features**, which may be **redundant** and lead to **multicollinearity** — a problem especially for linear models.

Correlation measures the **linear relationship** between variables.

- **Range:** -1 to $+1$
- **+1:** Strong positive correlation
- **-1:** Strong negative correlation
- **0:** No linear relationship

1. Import Libraries & Load Data

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
# Sample dataset
df = sns.load_dataset('iris')
df = df.drop('species', axis=1) # Drop target for feature correlation
```

2. Compute the Correlation Matrix

```
corr_matrix = df.corr(numeric_only=True)
```

3. Plot the Correlation Heatmap

```
plt.figure(figsize=(8, 6))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt=".2f", linewidths=0.5)
plt.title("Feature Correlation Heatmap")
plt.show()
```

4. Identify Highly Correlated Features

Features with correlation > 0.8 or 0.9 are often considered redundant.

```
# Get upper triangle of correlation matrix
upper = corr_matrix.where(
    np.triu(np.ones(corr_matrix.shape), k=1).astype(bool)
)

# Find features with correlation > 0.8
to_drop = [column for column in upper.columns if any(upper[column] > 0.8)]
print("Drop these features due to high correlation:", to_drop)
```

5. Drop Redundant Features (Optional)

```
df_reduced = df.drop(columns=to_drop)
```

Note:

When to Use Correlation-Based Feature Selection

- For **linear models** (e.g., Linear Regression, Logistic Regression)
- To avoid **redundancy** and **overfitting**
- As a **preprocessing step** before PCA or model training

2. Chi-squared test

The Chi-Squared Test is a statistical test used for feature selection, especially when both the features and target variable are categorical.

```
In [43]: from sklearn.feature_selection import SelectKBest
from sklearn.feature_selection import chi2
bestfeatures = SelectKBest(score_func=chi2, k=10)
fit = bestfeatures.fit(x,y)
dfscores = pd.DataFrame(fit.scores_)
dfcolumns = pd.DataFrame(x.columns)
featureScores = pd.concat([dfcolumns,dfscores],axis=1)
featureScores.columns = ['Specs','Score']
featureScores
```

```
In [47]: featureScores
```

```
Out[47]:
```

	Specs	Score
0	City_Code	0.122643
1	Region_Code	74.805013
2	Accommodation_Type	0.756115
3	Reco_Insurance_Type	3.965972
4	Upper_Age	2.612166
5	Lower_Age	1.572930
6	Is_Spouse	0.632296
7	Health Indicator	0.453390
8	Holding_Policy_Duration	7.662646
9	Holding_Policy_Type	0.836189
10	Reco_Policy_Cat	1894.032997
11	Reco_Policy_Premium	9575.065324
12	diff_age	0.039347

- Higher Chi² score → Stronger relationship with the target.
- Drop features with very low scores.

Note:

Pros	Cons
Fast and easy to compute	Only for categorical data
Good for initial filtering	Doesn't handle feature interactions
Interpretable	Requires non-negative inputs

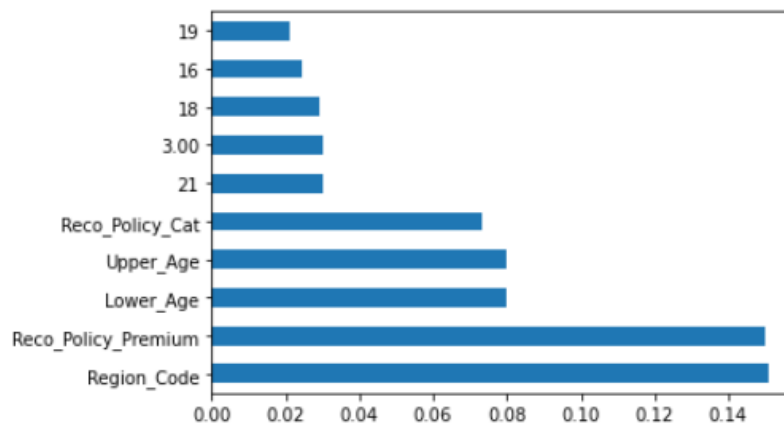
3.ExtraTreesClassifier method

ExtraTreesClassifier (Extremely Randomized Trees) is an ensemble learning method that can be used not only for classification but also for feature importance ranking and selection.

```
In [189]: from sklearn.ensemble import ExtraTreesClassifier
import matplotlib.pyplot as plt
model = ExtraTreesClassifier()
model.fit(x,y)
print(model.feature_importances_)
```

```
Out[189]: ExtraTreesClassifier()
```

```
In [192]: feat_importances = pd.Series(model.feature_importances_, index=x.columns)
feat_importances.nlargest(10).plot(kind='barh')
plt.show()
```



features with higher importance are more useful for prediction.

5.Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) is essential in the data science workflow because it lays the foundation for successful model building. Without a deep understanding of the data, models may perform poorly or produce misleading results.

1. Understand the Structure of the Data

This is the first and most important step of EDA. It helps you become familiar with the dataset you are working with before performing any analysis or building models.

- **Identify:**
 - **Know what your data contains**
 - **Check the shape and format**
 - **Identify variable types**
 - **Spot obvious issues (e.g., typos, wrong formats)**

Common Actions and Functions (using Python/pandas):

1. Load the Dataset

```
import pandas as pd  
df = pd.read_csv("your_file.csv")
```

2. View First and Last Few Rows

```
df.head()    # View first 5 rows  
df.tail()    # View last 5 rows
```

3. Check Dimensions

```
df.shape     # Returns (rows, columns)
```

4. Check Column Names

```
df.columns   # Lists all column names
```

5. Check Data Types

```
df.dtypes    # Shows data type of each column
```

6. General Info

```
df.info()
```

7. Basic Summary Statistics

```
df.describe()
```

2. Handle Missing Data

- **Identify missing values:**
 - `df.isnull().sum()`
- **Handle missing data by:**
 - Removing rows/columns with missing data
 - Imputing values (mean, median, mode, forward/backward fill)

3. Univariate Analysis

Univariate analysis involves examining **one variable at a time** to understand its distribution, central tendency, and spread.

It helps:

- Understand data types (categorical or numerical)
- Spot outliers, skewness, or unusual patterns
- Guide feature transformation or encoding

For Numerical Variables

1. Summary Statistics

```
df['age'].describe()
```

2. Visualizations

- **Histogram**

Shows the distribution of a variable.

```
import seaborn as sns
```

```
sns.histplot(df['age'], kde=True)
```

- **Boxplot**

Highlights outliers and spread.

```
sns.boxplot(x=df['age'])
```

- **Density Plot**

Smoothed version of a histogram.

```
sns.kdeplot(df['age'])
```

3. Check Skewness

```
df['age'].skew() # Positive or negative skew?
```

For Categorical Variables

1. Frequency Counts

```
df['gender'].value_counts()
```

2. Visualizations

- **Bar Plot**

```
sns.countplot(x='gender', data=df)
```

- **Pie Chart (optional)**

```
df['gender'].value_counts().plot.pie(autopct="%1.1f%%")
```

Why is Univariate Analysis Important?

- Helps identify:
 - Distribution type (normal, skewed, uniform)
 - Presence of **outliers or extreme values**
 - **Dominant categories** in categorical variables
- Guides:
 - **Imputation strategy** for missing values
 - **Feature scaling/encoding**
 - **Transformations** (e.g., log scaling)

4. Bivariate / Multivariate Analysis

Analysis Type	Description
Bivariate	Analyzing the relationship between two variables
Multivariate	Analyzing relationships among three or more variables

1. Bivariate Analysis

◇ Numerical vs Numerical

Scatter Plot

```
sns.scatterplot(x='height', y='weight', data=df)
```

Correlation Coefficient

```
df[['height', 'weight']].corr()
```

Heatmap

```
sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
```

◇ Numerical vs Categorical

Box Plot / Violin Plot

```
sns.boxplot(x='gender', y='salary', data=df)
```

Bar Plot (mean of numerical for each category)

```
df.groupby('gender')['salary'].mean().plot(kind='bar')
```

◇ Categorical vs Categorical

Cross Tabulation

```
pd.crosstab(df['gender'], df['purchased'])
```

Stacked Bar Chart

```
pd.crosstab(df['gender'], df['purchased']).plot(kind='bar', stacked=True)
```

2. Multivariate Analysis

Used to uncover deeper patterns or interactions across multiple variables.

◇ Pair Plot

Visualizes pairwise relationships between numerical variables.

```
sns.pairplot(df)
```

◇ GroupBy with Multiple Features

```
df.groupby(['gender', 'city'])['salary'].mean()
```

◇ Multivariate Visuals

Hue parameter in plots helps explore multiple features

```
sns.scatterplot(x='age', y='income', hue='gender', data=df)
```

◇ Correlation Matrix (for all numerical features)

```
sns.heatmap(df.corr(), annot=True)
```

Note:

Why This Step Matters:

- Detects **linear/nonlinear relationships** among variables
- Reveals **redundant or highly correlated** features (important for dimensionality reduction)
- Helps in **feature selection** and **feature interaction engineering**
- Prepares data for **model building** by understanding variable dependencies

5. Outlier Detection

Def: An outlier is a data point that significantly deviates from the rest of the data

It may result from:

- **Data entry errors**
- **Natural variability**
- **Rare or exceptional events**

Why Detect Outliers?

- **Statistical distortion:** Skews mean, variance, correlation, etc.
- **Model performance:** May reduce accuracy or increase overfitting
- **Insight discovery:** Outliers may represent fraud, anomalies, or rare conditions

♦ Common Methods to Detect Outliers:

1. Interquartile Range (IQR):

`Q1 = df['column'].quantile(0.25)`

`Q3 = df['column'].quantile(0.75)`

`IQR = Q3 - Q1`

`outliers = df[(df['column'] < Q1 - 1.5*IQR) | (df['column'] > Q3 + 1.5*IQR)]`

2. Using Z-Score

Z-score > 3 or < -3 is often considered an outlier.

```
from scipy.stats import zscore
```

```
import numpy as np
```

```
z_scores = np.abs(zscore(df['column']))
```

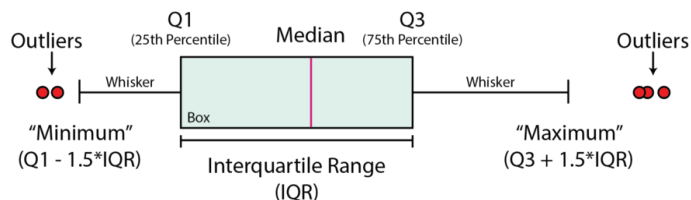
```
outliers = df[z_scores > 3]
```

2. Boxplot (Visualization)

```
import seaborn as sns
```

```
sns.boxplot(x=df['column'])
```

Outliers appear as dots outside the "whiskers"



4.Scatter Plot

Best for identifying outliers in relationships between two variables.

```
sns.scatterplot(x='height', y='weight', data=df)
```

6. Feature Correlation Analysis

- Use correlation matrix to identify highly correlated features
- Tools:
 - `sns.heatmap(df.corr(), annot=True)`
- Helps in **feature selection** and reducing multicollinearity

7. Data Visualization

- Visualize insights using:
 - **Seaborn**: barplot, boxplot, heatmap, pairplot
 - **Matplotlib**: line plots, histograms, pie charts
- Helps communicate data patterns clearly

8. Check for Data Imbalance

Data imbalance occurs when the target classes in a classification problem are not represented equally. This is **especially critical in binary and multi-class classification tasks**.

Why Check for Data Imbalance?

- Imbalanced data can lead to **biased models** that favor the majority class.
- Metrics like **accuracy can be misleading** (e.g., 95% accuracy when a model always predicts the majority class).

How to Check for Imbalance

Step 1: Use `value_counts()` to inspect the target distribution

```
import pandas as pd  
  
# Example target column  
df['target'].value_counts()
```

Step 2: Visualize the class distribution

```
import seaborn as sns  
  
import matplotlib.pyplot as plt  
  
sns.countplot(x='target', data=df)  
  
plt.title('Target Class Distribution')  
  
plt.show()
```

Step 3: Calculate class percentages

```
df['target'].value_counts(normalize=True) * 100
```

o/p

Class 0: 950 (95%)

Class 1: 50 (5%)

This is a **highly imbalanced dataset**, where Class 1 may be under-represented.

How to Handle Imbalanced Data

If imbalance is found, you can handle it using:

Resampling Techniques

- **Oversampling:** e.g., SMOTE, ADASYN
- **Undersampling:** Randomly reduce majority class samples

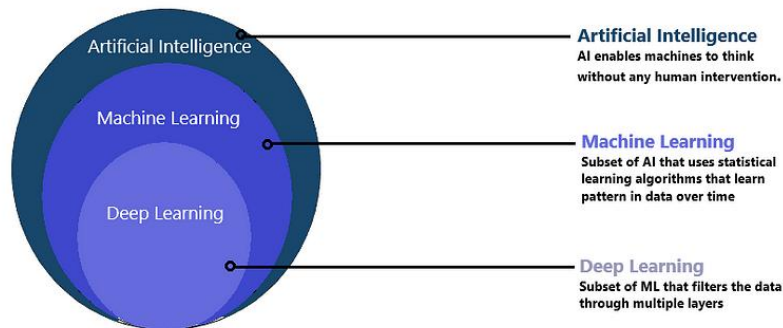
Use Model-Specific Methods

- Algorithms like **XGBoost**, **Random Forest**, and **Logistic Regression** can handle imbalance using parameters like:
 - `class_weight='balanced'`

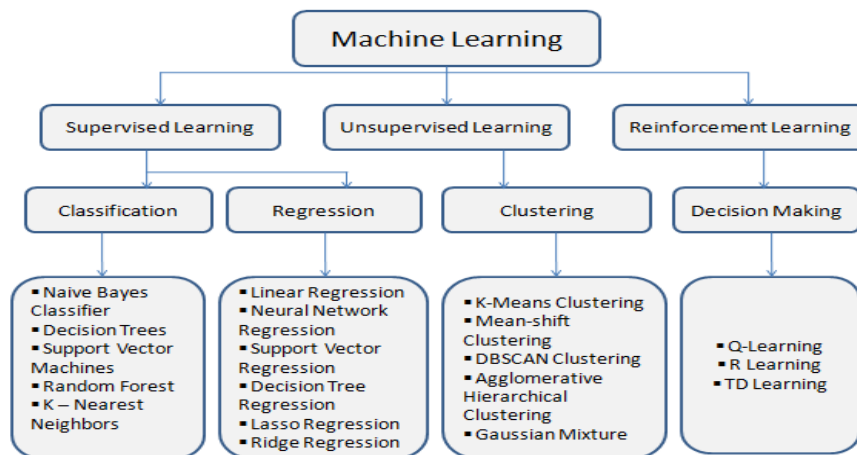
Evaluation Metrics for Imbalanced Data

-
- Precision, Recall
 - F1-score
 - ROC-AUC
 - Confusion Matrix

Unit 5: Model Development & Evaluation & Deployment



Types of Machine Learning



Supervised Learning is a type of machine learning where the model is **trained on a labeled dataset** — that means **input data (features)** are provided along with the **correct output (label or target)**.

Supervised Learning is of two types:

1. **Classification:** In Classification, a computer program is trained on a training dataset, and based on the training it categorizes the data in different class labels. This algorithm is used to predict the discrete values such as male|female, true|false, spam|not spam, etc.

Types of Classification Algorithms:

- Naive Bayes classifier

-
- Decision Trees
 - Logistic Regression
 - K-Nearest Neighbours
 - Support vector machine
 - Random forest classification

b) Regression: The task of the regression algorithm is to find the mapping function to map input variables(x) to the continuous output variable(y). Regression algorithms are used to predict continuous values such as price, salary, age, marks, etc.

Types of Regression Algorithms:

- Simple linear Regression
- Multiple linear Regression
- polynomial Regression
- Decision Tree Regression
- Random forest Regression
- Ensemble Method

2. Unsupervised Learning: Unsupervised Learning is a type of machine learning where the algorithm is given data without any labels, and the goal is to find hidden patterns or structure in the input data.

Types of Unsupervised Learning:

- Clustering
- Anomaly detection
- Association
- Autoencoders
- Latent variable models
- Neural Networks

3. Reinforcement Learning (RL): Reinforcement Learning is a type of machine learning where an agent learns to make decisions by interacting with an environment to maximize cumulative rewards.

1. Regression

1. Simple Linear Regression

Definition:

Simple Linear Regression is a statistical method used to model the relationship between two variables:

- **One independent variable (X)** – predictor or input.
- **One dependent variable (Y)** – response or output.

It assumes a linear relationship between the variables.

Equation of Simple Linear Regression:

The relationship is modeled using a **straight line**:

$$Y = a + bX$$

Where:

- a = intercept
- b = slope of the line
- X = independent variable
- Y = predicted dependent variable

Predict a student's score based on how many hours they studied.

Hours (X) Score (Y)

2	50
4	60
6	70
8	80

Hours (X) Score (Y)

10 90

1. Import Libraries

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
```

2.Prepare Data

```
X = np.array([2, 4, 6, 8, 10]).reshape(-1, 1) # Independent variable
Y = np.array([50, 60, 70, 80, 90])          # Dependent variable
```

3. Create and Train Model

```
model = LinearRegression()
model.fit(X, Y)
```

4. Print Coefficients

```
print("Intercept (a):", model.intercept_)
print("Slope (b):", model.coef_[0])
```

5. Make Prediction

```
predicted_score = model.predict([[7]]) # Predict score for 7 hours of study
print("Predicted Score for 7 hours:", predicted_score[0])
```

6. Visualize

```
plt.scatter(X, Y, color='blue', label='Actual Data')
plt.plot(X, model.predict(X), color='red', label='Regression Line')
plt.xlabel("Hours Studied")
plt.ylabel("Score")
plt.title("Simple Linear Regression Example")
plt.legend()
plt.show()
```

2. Multiple Linear Regression (MLR)

Def: Multiple Linear Regression is an extension of Simple Linear Regression where two or more independent variables ($X_1, X_2, \dots, X_n$) are used to predict the dependent variable (Y).

The general equation is:

$$Y = a + b_1X_1 + b_2X_2 + \dots + b_nX_n$$

Where:

- Y = dependent variable
- $X_1, X_2, \dots, X_n$ = independent variables
- a = intercept
- $b_1, b_2, \dots, b_n$ = coefficients

Example: Predict Student Score Based on Study Hours and Number of Practice

Tests

🎯 Problem:

Predict a student's **exam score** using:

- **Hours studied (X_1)**
- **Number of practice tests taken (X_2)**

Hours (X₁) Practice Tests (X₂) Score (Y)

2	1	50
4	2	60
6	2	65
8	3	80
10	4	90

Python Implementation Using sklearn**1. Import Libraries**

```
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression
import matplotlib.pyplot as plt
```

2. Prepare Data

```
# Features: Hours studied and Practice Tests
X = np.array([[2,1], [4,2], [6,2], [8,3], [10,4]])

# Target: Exam Score
Y = np.array([50, 60, 65, 80, 90])
```

3. Train Model

```
model = LinearRegression()
model.fit(X, Y)
```

4. Print Coefficients

```
print("Intercept (a):", model.intercept_)
```

```
print("Coefficient for Hours (b1):", model.coef_[0])  
print("Coefficient for Practice Tests (b2):", model.coef_[1])
```

5. Make Prediction

```
# Predict score for a student who studied 7 hours and took 3 practice tests  
prediction = model.predict([[7, 3]])  
print("Predicted Score:", prediction[0])
```

Output

Assume the model outputs:

- Intercept (a): **40.0**
- Coefficient for Hours: **4.5**
- Coefficient for Practice Tests: **5.0**

The equation becomes:

$$\text{Score} = 40 + 4.5 \cdot \text{Hours} + 5.0 \cdot \text{PracticeTests}$$

For 7 hours & 3 tests:

$$\text{Score} = 40 + 4.5(7) + 5(3) = 40 + 31.5 + 15 = 86.5$$

3. Polynomial Regression

Polynomial Regression is a type of regression that models the relationship between the independent variable X and the dependent variable Y as an n th-degree polynomial. It is used when the data shows a curved (non-linear) relationship.

Polynomial Regression Equation (Degree 2):

$$Y = a + b_1X + b_2X^2$$

Example: Predict Salary Based on Experience (Non-linear Relationship)**Sample Dataset****Experience (Years) (X) Salary (Y)**

1	45,000
2	50,000
3	65,000
4	85,000
5	110,000
6	150,000

From the table, salary increases faster at higher experience — not a straight line ⇒ non-linear relationship.

1. Import Libraries

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
```

2. Prepare Data

```
X = np.array([1, 2, 3, 4, 5, 6]).reshape(-1, 1)
Y = np.array([45000, 50000, 65000, 85000, 110000, 150000])
```

3. Transform Features to Polynomial

```
poly = PolynomialFeatures(degree=2)
X_poly = poly.fit_transform(X)
```

4. Train the Model

```
model = LinearRegression()
model.fit(X_poly, Y)
```

5. Make Prediction

```
# Predict salary for 3.5 years of experience
experience = np.array([[3.5]])
experience_poly = poly.transform(experience)
predicted_salary = model.predict(experience_poly)
print("Predicted Salary:", predicted_salary[0])
```

6. Plot the Curve

```
plt.scatter(X, Y, color='blue', label='Actual Data')
plt.plot(X, model.predict(X_poly), color='red', label='Polynomial Curve')
plt.xlabel("Years of Experience")
plt.ylabel("Salary")
plt.title("Polynomial Regression Example")
plt.legend()
plt.show()
```

Output (Example)

If the model learns the curve correctly, it may produce a prediction like:

- **Predicted Salary for 3.5 years = ₹75,000**

4. Decision Tree Regression

A Decision Tree Regressor is a type of regression model that splits the data into branches to predict continuous output values. Unlike linear models, it doesn't assume any linear relationship between input and output.

Characteristics:

- **Non-linear** model.
- Splits data into regions using decision rules.
- Captures complex relationships and interactions.
- Easy to visualize and interpret.

Implementation Decision Tree Regression

1.Import libraries and dataset

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeRegressor
```

```
# Sample dataset
```

```
data = {'Experience': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
        'Salary': [30000, 35000, 40000, 45000, 50000, 60000, 65000, 70000, 85000,
100000]}
df = pd.DataFrame(data)
```

2.Split into Features and Target

```
X = df[['Experience']] # Feature
```

```
y = df['Salary']    # Target
```

3. Train the Decision Tree Regressor

```
model = DecisionTreeRegressor()  
model.fit(X, y)
```

4. Make Predictions

```
# Predict salary for 6.5 years of experience  
experience = [[6.5]]  
predicted_salary = model.predict(experience)  
print(f"Predicted Salary for 6.5 years: ₹{predicted_salary[0]:.0f}")
```

Plotting the Decision Tree Regression Curve

```
# High-resolution input grid  
X_grid = np.arange(min(X['Experience']), max(X['Experience'])+0.1, 0.01)  
X_grid = X_grid.reshape(-1, 1)  
  
# Plot  
plt.scatter(X, y, color='red', label="Actual")  
plt.plot(X_grid, model.predict(X_grid), color='blue', label="Prediction")  
plt.title("Decision Tree Regression")  
plt.xlabel("Years of Experience")  
plt.ylabel("Salary")  
plt.legend()  
plt.grid(True)  
plt.show()
```

Note

Decision Tree Regressor vs Linear Regression:

Feature	Decision Tree Regression	Linear Regression
Relationship	Non-linear	Linear
Feature Interaction	Automatically captured	Must be engineered manually
Interpretability	Easy (tree structure)	Easy (equation)
Overfitting	Can overfit (prone)	Less prone (if simple)

5.Random Forest Regression

Random Forest Regression is an **ensemble learning** method that uses **multiple Decision Trees** to predict continuous outcomes. Instead of relying on a single tree (like Decision Tree Regression), it builds many trees and averages their outputs for better accuracy and generalization.

How It Works:

1. **Multiple Decision Trees** are built using different subsets of the data and features (using **bootstrapping** and **feature randomness**).
2. Each tree makes its own **regression prediction**.
3. The final prediction is the **average of all tree predictions**.

Implementation of random forest regression

```
import numpy as np
import pandas as pd
from sklearn.ensemble import RandomForestRegressor
import matplotlib.pyplot as plt
```

```
# Sample dataset
```

```
X = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10]).reshape(-1, 1)
```

```
y = np.array([30000, 35000, 40000, 45000, 50000, 60000, 65000, 70000, 85000, 100000])
```

```
# Create and train the model
```

```
model = RandomForestRegressor(n_estimators=100, random_state=42)
```

```
model.fit(X, y)
```

```
# Predict salary for 6.5 years of experience
```

```
pred = model.predict([[6.5]])
```

```
print("Predicted salary for 6.5 years experience:", int(pred[0]))
```

Plotting the Curve

```
# High-resolution curve
```

```
X_grid = np.arange(1, 10.1, 0.01).reshape(-1, 1)
```

```
plt.scatter(X, y, color='red', label="Actual")
```

```
plt.plot(X_grid, model.predict(X_grid), color='blue', label="Random Forest Prediction")
```

```
plt.title("Random Forest Regression")
```

```
plt.xlabel("Experience (Years)")
```

```
plt.ylabel("Salary")
```

```
plt.legend()
```

```
plt.grid(True)
```

```
plt.show()
```

2. Classification

1. k-NN Classification

k-Nearest Neighbors (k-NN) is a supervised machine learning algorithm used for classification and regression. In classification, it predicts the class of a new data point by looking at the 'k' closest labeled data points in the training set.

How It Works:

1. **Choose k** = number of neighbors.
2. Calculate the **distance** between the test data point and all training points.
3. Select the **k closest** data points.
4. Perform a **majority vote** on the labels of the k neighbors.
5. Assign the class with the **most votes**.

Common Distance Metrics:

Euclidean distance (default):

$$d = \sqrt{\sum (x_i - y_i)^2}$$

Manhattan distance

Minkowski distance

```
# Import necessary libraries
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report

# Step 1: Load dataset
iris = load_iris()
X = iris.data      # Features
y = iris.target    # Labels (0,1,2 for Setosa, Versicolor, Virginica)

# Step 2: Split data (70% train, 30% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Step 3: Feature scaling (important for k-NN)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Step 4: Train k-NN classifier with k=3
knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train, y_train)
```

```
# Step 5: Make predictions
```

```
y_pred = knn.predict(X_test)
```

```
# Step 6: Evaluate the model
```

```
print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
print("Classification Report:")
```

```
print(classification_report(y_test, y_pred, target_names=iris.target_names))
```

o/p

Accuracy: 1.0

Classification Report:

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	16
versicolor	1.00	1.00	1.00	16
virginica	1.00	1.00	1.00	13

2. Support Vector Machines (SVM)

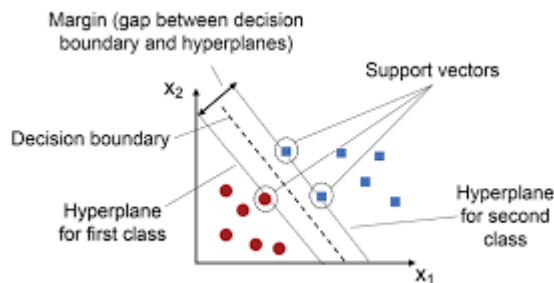
Support Vector Machine (SVM) is a **supervised machine learning algorithm** used primarily for **classification**, and also for regression. It finds an optimal decision boundary (**hyperplane**) that best separates data into different **classes**.

How SVM Works (Conceptually)

- SVM finds the **maximum-margin hyperplane** that separates classes.
- **Support Vectors**: Data points **closest** to the hyperplane.
- The model tries to **maximize the margin** between classes for better generalization.

Linear vs Non-Linear:

- If data is **linearly separable**, SVM uses a straight line (or hyperplane).
- For **non-linear data**, SVM uses the **kernel trick** to transform data into higher dimensions where a linear separator is possible.



Types of Kernels in SVM

Kernel	Use Case
linear	Linearly separable data
rbf	Gaussian kernel – for non-linear data
poly	Polynomial boundary
sigmoid	S-shaped boundaries (rarely used)

Implementation of SVM

```
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report

# Load Iris dataset
iris = datasets.load_iris()
X = iris.data
y = iris.target

# Split into train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Standardize the features (important for SVM)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Create and train the SVM model (with RBF kernel)
model = SVC(kernel='rbf', C=1.0, gamma='scale')
model.fit(X_train, y_train)

# Predict on test set
y_pred = model.predict(X_test)
```

```
# Evaluate the model
```

```
print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
print("Classification Report:\n", classification_report(y_test, y_pred,  
target_names=iris.target_names))
```

Output

Accuracy: 1.0

Classification Report:

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	16
versicolor	1.00	1.00	1.00	14
virginica	1.00	1.00	1.00	15

3. Logistic Regression

Logistic Regression is a classification algorithm, not a regression one.

It is used to predict categorical outcomes – typically binary classification (yes/no, 0/1, spam/not spam).

Key Idea:

- Instead of predicting a continuous value, **Logistic Regression predicts probabilities.**
- It uses the **logistic (sigmoid) function** to map any real number into a range between **0 and 1**.

Sigmoid Function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Where $z = w_1 * x_1 + w_2 * x_2 + \dots + b$

Implementation Logistic Regression (Iris Dataset)

```
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Load dataset
iris = datasets.load_iris()
X = iris.data
y = (iris.target == 0).astype(int) # 1 if Setosa, 0 otherwise

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Standardize features
scaler = StandardScaler()
```

```
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Train Logistic Regression model
model = LogisticRegression()
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Evaluate
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test, y_pred))
```

o/p

Accuracy: 1.0

Confusion Matrix:

[[29 0]

[0 16]]

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	29
1	1.00	1.00	1.00	16

4. Decision Tree Classification

Decision Tree Classification is a **supervised learning algorithm** used for classifying data into categories by **splitting it based on decision rules** derived from feature values.

It creates a tree-like model of decisions:

- **Nodes** represent features.
- **Branches** represent conditions.
- **Leaves** represent class labels (predictions).

How it Works:

1. The algorithm selects the **best feature** to split the data (using Gini Impurity or Entropy).
2. Splits the dataset into subsets.
3. Repeats the process recursively for each subset.
4. The process stops when a stopping criterion is met (e.g., max depth or pure node).

Implementation of Decision Tree Classification

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report, plot_tree
import matplotlib.pyplot as plt

# Step 1: Load dataset
iris = load_iris()
X = iris.data
y = iris.target
```

Step 2: Split dataset into train and test

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

Step 3: Train Decision Tree Classifier

```
model = DecisionTreeClassifier(criterion='gini', max_depth=3, random_state=42)
```

```
model.fit(X_train, y_train)
```

Step 4: Predict

```
y_pred = model.predict(X_test)
```

Step 5: Evaluate

```
print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
print("Classification Report:\n", classification_report(y_test, y_pred,  
target_names=iris.target_names))
```

Step 6: Visualize the tree

```
plt.figure(figsize=(12, 8))
```

```
plot_tree(model, filled=True, feature_names=iris.feature_names,  
class_names=iris.target_names)
```

```
plt.title("Decision Tree for Iris Classification")
```

```
plt.show()
```

Output

Accuracy: 1.0

Classification Report:

```
precision  recall  f1-score  support
```

setosa	1.00	1.00	1.00	16
versicolor	1.00	1.00	1.00	14
virginica	1.00	1.00	1.00	15

5.Random Forest Classification

Random Forest is an ensemble learning method used for classification and regression. In classification, it builds multiple decision trees and outputs the majority class (mode) of their predictions.

How It Works:

1. It creates many Decision Trees using:
 - Random subsets of the data (bootstrapping)
 - Random subsets of features
2. Each tree makes a prediction.
3. The final prediction is the majority vote from all trees.

This helps reduce overfitting and improves accuracy and robustness.

Implementation of Random Forest

```
from sklearn.datasets import load_iris
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report

# Step 1: Load dataset
iris = load_iris()
```

```
X = iris.data
y = iris.target

# Step 2: Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Step 3: Train Random Forest model
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# Step 4: Predict
y_pred = model.predict(X_test)

# Step 5: Evaluate
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test, y_pred,
target_names=iris.target_names))
```

Output

Accuracy: 1.0

Classification Report:

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	16
versicolor	1.00	1.00	1.00	14
virginica	1.00	1.00	1.00	15

Key Parameters:

Parameter	Description
n_estimators	Number of trees in the forest
max_depth	Max depth of each tree
criterion	'gini' or 'entropy' for split quality
max_features	Number of features considered per split

3.Unsupervised Learning

Clustering Algorithms:

- K-Means Clustering
- Hierarchical Clustering
- DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

1. K-Means Clustering

K-Means Clustering is one of the most popular **unsupervised learning algorithms** used to group data into **K number of clusters**, where each data point belongs to the cluster with the **nearest mean** (called **centroid**).

How It Works (Step-by-Step)

1. **Choose the number of clusters KKK.**
2. **Initialize K random centroids.**
3. **Assign** each data point to the **nearest centroid** (based on distance).
4. **Update** centroids by calculating the **mean** of all points in each cluster.
5. **Repeat** steps 3 and 4 until centroids **stop changing significantly** (convergence).

Implementation K Means

```
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs

# Generate sample data
X, _ = make_blobs(n_samples=300, centers=4, cluster_std=0.60, random_state=0)

# Apply KMeans
kmeans = KMeans(n_clusters=4)
kmeans.fit(X)
y_kmeans = kmeans.predict(X)

# Plot the clusters
plt.scatter(X[:, 0], X[:, 1], c=y_kmeans, s=50, cmap='viridis')

# Plot the centroids
centers = kmeans.cluster_centers_
plt.scatter(centers[:, 0], centers[:, 1], c='red', s=200, alpha=0.75, marker='X')
plt.title("K-Means Clustering")
plt.show()
```

Choosing K – The Elbow Method

To find the best value of **K**:

1. Run K-Means for different K values.
2. Plot the **inertia** (sum of squared distances from points to their cluster center).
3. Look for the "elbow" point where inertia stops decreasing sharply.

```
inertia = []  
K_range = range(1, 10)  
  
for k in K_range:  
    km = KMeans(n_clusters=k)  
    km.fit(X)  
    inertia.append(km.inertia_)  
  
plt.plot(K_range, inertia, 'bo-')  
plt.xlabel('Number of clusters (K)')  
plt.ylabel('Inertia')  
plt.title('Elbow Method')  
plt.show()
```

Applications of K-Means

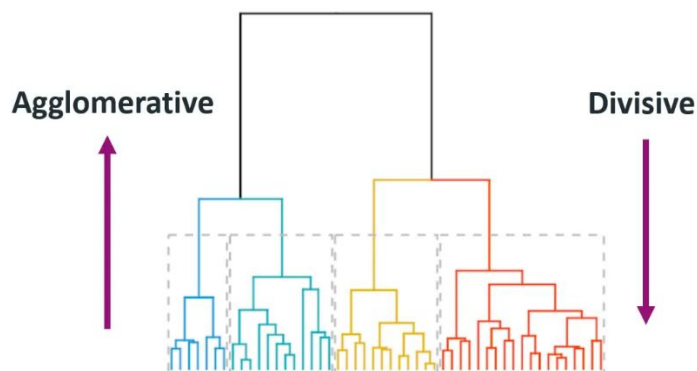
- **Customer segmentation** in marketing
- **Image compression** (by reducing color palettes)
- **Document or topic clustering**
- **Anomaly detection**

2. Hierarchical Clustering

Hierarchical Clustering is an **unsupervised learning** technique that builds a **hierarchy of clusters**. Unlike K-Means, it doesn't require specifying the number of clusters (**K**) in advance.

Types of Hierarchical Clustering

1. **Agglomerative** (Bottom-Up – Most common):
Start with **each point as its own cluster**, then **merge** the closest clusters step by step.
2. **Divisive** (Top-Down):
Start with **all points in one cluster**, then **split** recursively.



How Agglomerative Clustering Works:

1. Assign each data point to its own cluster.
2. Find the **closest pair** of clusters and **merge** them.
3. Repeat step 2 until only one cluster (the root of the hierarchy) remains.
4. Use a **dendrogram** (tree diagram) to decide where to cut and form final clusters.

Linkage Criteria (How distance between clusters is calculated):

Method	Description
Single Link	Minimum distance between points in two clusters
Complete Link	Maximum distance between points in two clusters
Average Link	Average distance between all point pairs
Ward's Method	Minimizes the variance within each cluster

Implementation Agglomerative Clustering

```
from sklearn.datasets import make_blobs
from scipy.cluster.hierarchy import dendrogram, linkage
from sklearn.cluster import AgglomerativeClustering
import matplotlib.pyplot as plt
```

Sample Data

```
X, _ = make_blobs(n_samples=150, centers=3, random_state=42)
```

Plot Dendrogram

```
linked = linkage(X, method='ward') # 'ward' minimizes within-cluster variance
plt.figure(figsize=(10, 5))
dendrogram(linked, truncate_mode='lastp', p=30)
plt.title("Dendrogram (Ward Linkage)")
plt.xlabel("Data Points")
plt.ylabel("Distance")
plt.show()
```

Apply Agglomerative Clustering

```
hc = AgglomerativeClustering(n_clusters=3, affinity='euclidean', linkage='ward')
labels = hc.fit_predict(X)
```

```
# Visualize Clusters
plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis')
plt.title("Agglomerative Clustering")
plt.show()
```

Deep Learning

Deep Learning is a subset of Machine Learning that uses neural networks with many layers (deep neural networks) to model complex patterns in data. It's the driving force behind applications like image recognition, speech processing, natural language understanding, and autonomous vehicles.

Application

Field	Use
Computer Vision	Face detection, object recognition, medical imaging
NLP	Chatbots, translation, sentiment analysis
Speech	Voice assistants, transcription
Healthcare	Disease prediction, drug discovery
Finance	Fraud detection, algorithmic trading
Autonomous Vehicles	Lane detection, object avoidance

Core Components in Deep Learning

1. Artificial Neural Networks (ANN)

An Artificial Neural Network (ANN), often referred to as a neural network, is a computational model inspired by the human brain's neural structure. ANNs consist of interconnected nodes, or "neurons," organized in layers. Each neuron processes and transmits information to other neurons, enabling the network to learn patterns and make decisions. ANNs are a cornerstone of machine learning, particularly in tasks that involve pattern recognition, classification, regression, and more complex computations.

Architecture of Artificial Neural Networks

An ANN typically comprises three types of layers:

1. Input Layer

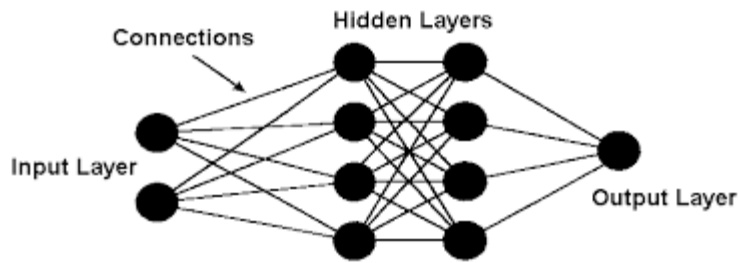
The initial layer that receives input data, which could be features from a dataset.

2. Hidden Layers

These intermediate layers process data and learn intricate patterns. A neural network can consist of multiple hidden layers, making it "deep" (Deep Neural Network, or DNN).

3. Output Layer

The final layer that produces the network's prediction or output.



Activation function

Function	Range	Non-linear	Use in
Sigmoid	$(0, 1)$	☒	Output layer (binary)
Tanh	$(-1, 1)$	☒	Hidden layers
ReLU	$[0, \infty)$	☒	Hidden layers
Leaky ReLU	$(-\infty, \infty)$	☒	Hidden layers
Softmax	$(0, 1)$	☒	Output layer (multi-class)

Working of Artificial Neural Networks

The operation of Artificial Neural Networks (ANNs) is inspired by the way biological neurons work in the human brain. ANNs consist of layers of interconnected nodes, or artificial neurons, each with a set of weights and biases. The working of ANNs involves the following steps:

1. Input Layer

The input layer receives raw data or features from the input source. Each input is associated with a weight, indicating its importance.

2. Hidden Layers

In the hidden layers, each neuron processes the weighted sum of inputs and biases using an activation function. The activation function introduces non-linearity, allowing the network to capture complex patterns.

3. Weights and Biases Adjustment

During the learning phase, the network adjusts weights and biases through a process called [back propagation](#). It compares the network's output to the desired output, calculates the error, and adjusts weights to minimize this error.

4. Output Layer

The processed information propagates through the hidden layers to the output layer. The output layer provides the final prediction or classification result.

5. Training

The network iteratively updates weights and biases using training data to minimize the prediction error. This process fine-tunes the network's ability to make accurate predictions.

6. Prediction

Once trained, the network can process new, unseen data and generate predictions or classifications based on the patterns it has learned.

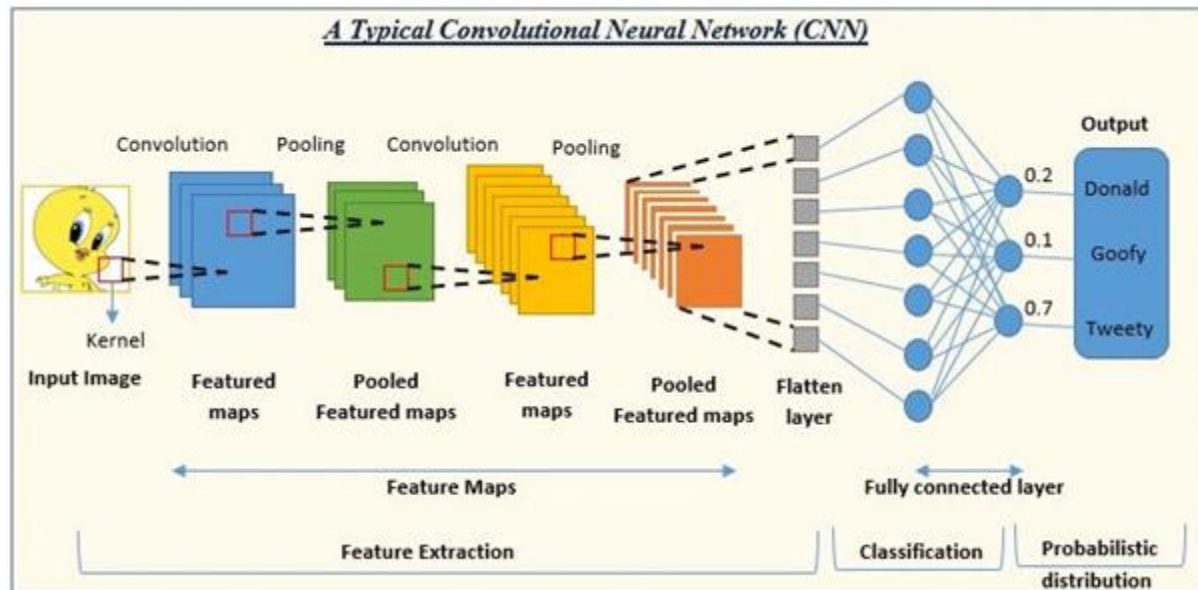
Types of ANN

Architecture	Use Case
MLP (Multilayer Perceptron)	Tabular data
CNN (Convolutional Neural Network)	Image classification, object detection
RNN (Recurrent Neural Network)	Sequential data (e.g., time series, text)
LSTM / GRU	Improved RNNs for longer memory
Transformer	NLP, vision (e.g., BERT, GPT, ViT)
GAN (Generative Adversarial Network)	Image generation, style transfer
Autoencoders	Dimensionality reduction, anomaly detection

Convolutional Neural Network (CNN)

It is a type of deep learning model primarily used for processing data with a grid-like topology, such as images. CNNs are highly effective for image classification, object detection, and computer vision tasks due to their ability to automatically and adaptively learn spatial hierarchies of features.

Architecture of a CNN



A typical CNN consists of the following layers:

1. Input Layer

- Accepts the raw image as input (e.g., 28x28x1 for grayscale, 224x224x3 for RGB).

2. Convolutional Layer

- Applies filters (kernels) to extract features like edges, textures, shapes.
- Output = Feature maps.
- Key parameters: filter size, stride, padding.

3. Activation Function

- Usually ReLU (Rectified Linear Unit) is used to introduce non-linearity.

4. Pooling Layer (Subsampling/Downsampling)

- Reduces spatial size to decrease computation and overfitting.
- Common: Max Pooling, Average Pooling.

5. Fully Connected (Dense) Layer

- Neurons are fully connected to the previous layer.
- Works like a traditional neural network classifier.

6. Output Layer

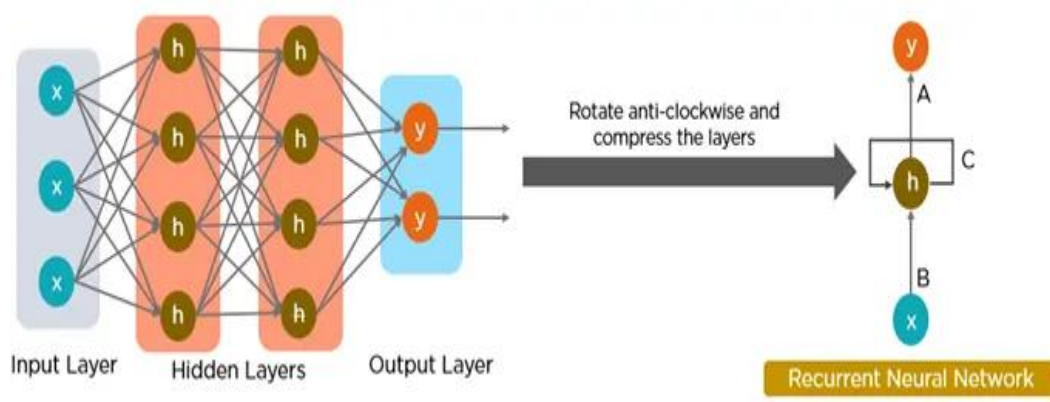
- Provides the final class probabilities (using softmax for multi-class classification)

Applications of CNNs

- Image classification (e.g., cats vs dogs)
- Object detection (YOLO, SSD)
- Face recognition
- Medical image analysis
- Handwriting recognition (e.g., MNIST)
- Self-driving cars (lane and object detection)

Recurrent Neural Network (RNN)

RNN (Recurrent Neural Network) is a type of neural network designed for sequential data. Unlike traditional feedforward networks, RNNs have "memory" — they use previous outputs as inputs for the current step, making them ideal for tasks involving time-series, text, speech, or any sequence-based data.



RNN Architecture

RNNs are a type of neural network that have hidden states and allow past outputs to be used as inputs. They usually follow a certain architecture. One example is the Deep RNN: by stacking multiple RNN layers on top of each other, deep RNNs create a more complex architecture. This allows them to capture intricate relationships within very long sequences of data. They are particularly useful for tasks where the order of elements spans long stretches.

Here's a breakdown of its key components:

- **Input Layer:** This layer receives the initial element of the sequence data. For example, in a sentence, it might receive the first word as a vector representation.
- **Hidden Layer:** The heart of the RNN, the hidden layer contains a set of interconnected neurons. Each neuron processes the current input along with the information from the previous hidden layer's state. This "state" captures the network's memory of past inputs, allowing it to understand the current element in context.
- **Activation Function:** This function introduces non-linearity into the network, enabling it to learn complex patterns. It transforms the combined input from the current input layer and the previous hidden layer state before passing it on.
- **Output Layer:** The output layer generates the network's prediction based on the processed information. In a language model, it might predict the next word in the sequence.
- **Recurrent Connection:** A key distinction of RNNs is the recurrent connection within the hidden layer. This connection allows the network to pass the hidden state information (the network's memory) to the next time step. It's like passing a baton in a relay race, carrying information about previous inputs forward

Types of RNN

Type	Description
Vanilla RNN	Basic RNN structure. Suffers from long-term memory loss (vanishing gradients).
LSTM	Long Short-Term Memory — solves vanishing gradient problem, remembers long dependencies.
GRU	Gated Recurrent Unit — simpler and faster than LSTM but performs comparably.

Applications of RNNs

- Text generation (language modeling, next word prediction)
- Machine translation (seq2seq models)
- Speech recognition
- Stock market prediction
- Music generation

- Time-series forecasting

LSTM (Long Short-Term Memory)

LSTM is a special type of RNN (Recurrent Neural Network) designed to remember long-term dependencies and solve the vanishing gradient problem that standard RNNs suffer from. It does this using a cell state and gates to control the flow of information.

LSTM Architecture Components

At each time step, the LSTM cell takes:

- Input x_t
- Previous hidden state h_{t-1}
- Previous cell state c_{t-1}

It outputs:

- New hidden state h_t
- New cell state c_t

Main Components of LSTM Cell

Gate	Function	Equation
<u>Forget Gate</u>	<u>Decides what to forget from previous cell state</u>	$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$
<u>Input Gate</u>	<u>Decides what new info to add</u>	$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$
<u>Candidate (Update)</u>	<u>Creates new candidate values</u>	$\tilde{c}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c)$
<u>Cell State Update</u>	<u>Updates memory</u>	$c_t = f_t * c_{t-1} + i_t * \tilde{c}_t$
<u>Output Gate</u>	<u>Decides what to output</u>	$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$
	<u>Final output hidden state</u>	$h_t = o_t * \tanh(c_t)$

Applications of LSTM

- Text generation (e.g., autocomplete)
- Speech recognition
- Machine translation
- Stock price prediction
- Video classification

Model evaluation Metrics

Model evaluation metrics are used to measure the performance of machine learning models. The choice of metrics depends on the type of problem: **classification**, **regression**, or others like **clustering**.

1. Classification Metrics

- **TP (True Positive):** Correctly predicted positive cases
 - **TN (True Negative):** Correctly predicted negative cases
 - **FP (False Positive):** Incorrectly predicted as positive
 - **FN (False Negative):** Incorrectly predicted as negative
-

Common Metrics:

Metric	Formula	Description
Accuracy	$(TP + TN) / (TP + TN + FP + FN)$	Overall correctness
Precision	$TP / (TP + FP)$	How many predicted positives are actual positives
Recall (Sensitivity)	$TP / (TP + FN)$	How many actual positives are correctly predicted
F1 Score	$2 * (Precision * Recall) / (Precision + Recall)$	Harmonic mean of precision and recall
Specificity	$TN / (TN + FP)$	Correct identification of negatives
AUC-ROC	Area under the ROC Curve	Measures ability to distinguish classes
Log Loss	$-1/N * \sum [y \log(p) + (1 - y) \log(1 - p)]$	Penalizes confident but incorrect predictions

3. Regression Metrics

Metric	Formula	Description
MAE (Mean Absolute Error)	$(1/n) \sum$	$y_i - \hat{y}_i$
MSE (Mean Squared Error)	$(1/n) \sum (y_i - \hat{y}_i)^2$	Penalizes larger errors more
RMSE (Root Mean Squared Error)	$\sqrt{MSE}$	Same units as target
R² Score (Coefficient of Determination)	$1 - (SS_{res} / SS_{tot})$	Measures how well model fits the data
MAPE (Mean Absolute Percentage Error)	$(1/n) \sum$	$(y_i - \hat{y}_i) / y_i$

3. Clustering Metrics

Metric	Description
Silhouette Score	Measures how similar an object is to its own cluster vs. other clusters
Davies–Bouldin Index	Lower is better; evaluates intra-cluster similarity and inter-cluster differences
Adjusted Rand Index (ARI)	Compares clustering result with ground truth
Homogeneity, Completeness, V-measure	Evaluate quality when true labels are known

Model Deployment

Model Deployment is the process of integrating a trained machine learning model into a production environment where it can be used to make real-time or batch predictions. This is a crucial step in the ML lifecycle that enables your model to deliver business value.

Steps in Model Deployment

1. Train and Save the Model

- Use your preferred ML library (e.g., scikit-learn, TensorFlow, PyTorch).
- Save the trained model using:
 - joblib or pickle (for scikit-learn)
 - model.save() (for TensorFlow/Keras)
 - torch.save() (for PyTorch)

Example: Saving model in scikit-learn

```
import joblib
```

```
joblib.dump(model, 'model.pkl')
```

2. Create a Prediction API

Use a web framework like **Flask**, **FastAPI**, or **Django** to expose your model as a REST API.

Flask Example

```
from flask import Flask, request, jsonify
import joblib
model = joblib.load('model.pkl')
app = Flask(__name__)
@app.route('/predict', methods=['POST'])
def predict():
    data = request.get_json()
    prediction = model.predict([data['features']])
    return jsonify({'prediction': prediction.tolist()})
app.run(port=5000)
```

3. Containerize with Docker (Optional)

Docker helps in packaging the app and its dependencies into a container for consistent deployment.

Dockerfile

FROM python:3.10

WORKDIR /app

COPY . /app

RUN pip install -r requirements.txt

CMD ["python", "app.py"]

5. Deploy to a Platform

You can deploy your model API to:

- **Cloud Services**
 - AWS (SageMaker, EC2)
 - Google Cloud (AI Platform, Cloud Run)
 - Azure ML
- **PaaS**
 - Heroku
 - Render
 - Railway
- **On-Premises**
 - In-house servers using Docker or Kubernetes

5. Monitoring and Maintenance

- Track model performance using tools like **Prometheus**, **Grafana**, or **MLflow**.
- Watch for **model drift** or **data drift**.
- Retrain and redeploy models when necessary.

Natural Language Processing

Natural Language Processing (NLP) is a subfield of artificial intelligence (AI) that enables computers to understand, interpret, and generate human language.

Task	Description	Example
Tokenization	Split text into words, phrases, or symbols	"I love NLP" → ["I", "love", "NLP"]
POS Tagging	Assign parts of speech (noun, verb, etc.)	"run" → Verb
Named Entity Recognition (NER)	Detect names, places, organizations	"Sachin lives in Mumbai" → [Person, Location]
Sentiment Analysis	Detect emotional tone (positive/negative)	"This movie is great!" → Positive
Text Classification	Assign categories to text	Spam vs Not Spam
Machine Translation	Translate from one language to another	English → Hindi
Text Summarization	Generate shorter version of a long text	TL;DR of news
Question Answering	Provide answers from context or document	Q: "Who is CEO of Tesla?" → "Elon Musk"
Language Modeling	Predict the next word or generate text	"The cat sat on the ____" → "mat"

Techniques Used in NLP

- ◇ **Traditional (Rule-based & ML)**
 - **Bag of Words (BoW)**
 - **TF-IDF (Term Frequency-Inverse Document Frequency)**
 - **Naive Bayes, SVM, Logistic Regression**
- ◇ **Deep Learning**
 - **Word Embeddings:** Word2Vec, GloVe, FastText
 - **RNN, LSTM, GRU**
 - **Transformer Models:** BERT, GPT, RoBERTa